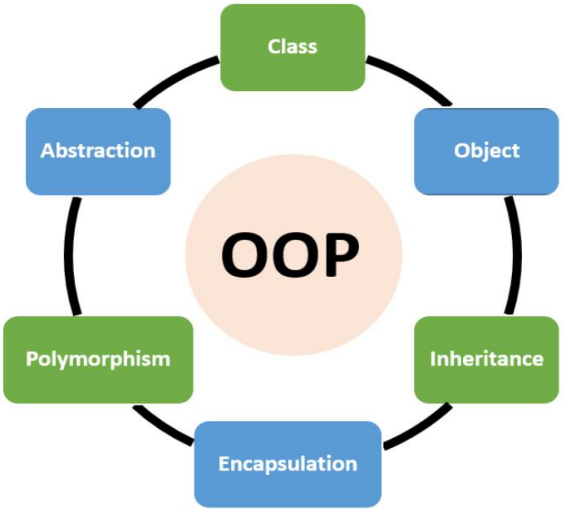
HADLA
MEDIA INFORMASI

BUKU AJAR :
PEMROGRAMAN BERORIENTASI OBJEK
MENGGUNAKAN VISUAL BASIC



NURHADI

2023

BUKU AJAR:
PEMROGRAMAN BERORIENTASI OBJEK MENGGUNAKAN VISUAL BASIC

Penulis

Nurhadi

PENERBIT:

HADLA
MEDIA INFORMASI

BUKU AJAR:
PEMROGRAMAN BERORIENTASI OBJEK MENGGUNAKAN VISUAL BASIC

Tim Penulis:
Nurhadi

Desain Cover:
Sulaiman

Tata Letak:
Sulaiman

ISBN:
-

Cetakan Pertama:
Agustus, 2023

Hak Cipta 2023, Pada Penulis

Hak Cipta Dilindungi Oleh Undang-Undang

Copyright © 2023
by HADLA Media Informasi
All Right Reserved

Dilarang keras menerjemahkan, memfotokopi, atau memperbanyak sebagian atau seluruh isi buku ini tanpa izin tertulis dari Penerbit.

PENERBIT:

HADLA
MEDIA INFORMASI

JL K.H. Agus Salim No. 30 RT 03 RW 08 Kel. Bekasi Jaya
Kec. Bekasi Timur Kota Bekasi, Provinsi Jawa Barat

Website: www.media.hadlacorp.com

PRAKATA

Bismillahirrahmanirrahim

Alhamdulillah penulisan buku ini sudah selesai, semoga bisa bermanfaat untuk orang banyak, dan bisa digunakan untuk mahasiswa dalam mempelajari pemrograman berorientasi objek. Buku ini dibuat sebagai bahan ajar untuk mata kuliah Pemrograman berorientasi objek dengan aplikasinya menggunakan visual basic. Dalam buku ini membahas secara rinci materi perihal pemrograman berorientasi objek mulai dari dasar sampai konsep pemrograman berorientasi objek pada dunia nyata.

Dalam buku ini, akan menjelaskan pengenalan tentang konsep pemrograman berorientasi objek, konsep dasar pemrograman berorientasi objek seperti Enkapsulasi, polimorfisme, Abstraksi dan lain sebagainya. Dalam buku ini juga akan menjelaskan visual basic sebagai aplikasi yang akan digunakan untuk mengimplementasikan materi-materi tersebut.

Pada akhir bab terdapat juga contoh pemrograman berorientasi objek pada dunia nyata. Sehingga mahasiswa mampu memahami materi lebih dalam lagi dari apa yang telah dipelajarinya. Dan akan dibahas juga pemrograman berorientasi objek yang lebih lanjut sehingga memberikan pengetahuan tambahan bagi mahasiswa yang mempelajari nya.

Diharapkan setelah mempelajari buku ini di harapakan mahasiswa dapat memahami dengan dalam konsep pemrograman berorientasi objek mulai dari dasar sampai konsep pemrograman berorientasi objek yang lebih lanjut.

Salam
Penulis

DAFTAR ISI

PRAKATA ..... **iii**
DAFTAR ISI ..... **v**
KEGIATAN BELAJAR I- PENDAHULUAN **1**
A. Pengenalan tentang konsep pemrograman Berorientasi objek 1
B. Keuntungan dan manfaat dari pemrograman berorientasi objek 3
C. Perbandingan antara pemrograman berorientasi objek dan paradigma lain 4
D. Penjelasan singkat tentang bahasa Visual Basic sebagai alat pengembangan 5
E. Rangkuman 7
F. Latihan 7
KEGIATAN BELAJAR II - KONSEP DASAR PEMROGRAMAN BERORIENTASI OBJEK **8**
A. Apa itu Objek, Kelas, dan Instance 8
B. Enkapsulasi: mengelompokkan data dan metode dalam kelas 10
C. Pewarisan: menciptakan hierarki kelas dan mewarisi sifat-sifat dari kelas yang lebih tinggi 11
D. Polimorfisme: penggunaan metode yang sama dengan perilaku yang berbeda 13
E. Abstraksi: menyembunyikan detail kompleks dari implementasi di luar dunia objek 14
G. Rangkuman 18
H. Latihan 19
KEGIATAN BELAJAR III – KELAS DAN OBJEK DALAM VISUAL BASIC **20**
A. Membuat Kelas Dalam Visual Basic 21
B. Membuat objek dari kelas yang telah dibuat 21
C. Mengakses property dan kelas 22
D. Mengelola aksesibilitas anggota kelas 24

E. Rangkuman 26
F. Latihan 26
KEGIATAN BELAJAR IV - PEWARISAN DAN POLIMORFISME 27
A. Menerapkan pewarisan untuk menciptakan hubungan antara kelas 27
B. Menggunakan metode yang diwarisi dan menggantikannya (overriding) 29
C. Menggunakan kata kunci Overrides dan NotOverridable 30
D. Rangkuman 32
E. Latihan 32
KEGIATAN BELAJAR V - ABSTRAKSI DAN ANTARMUKA 33
A. Menerapkan abstraksi untuk membuat kerangka kerja kelas umum 33
B. Membuat antarmuka (interface) untuk menentukan kontrak yang harus diikuti oleh kelas-kelas tertentu 35
C. Mengimplementasikan antarmuka dalam kelas 36
D. Rangkuman 38
E. Latihan 38
KEGIATAN BELAJAR VI - ENKAPSULASI DAN PENGENDALIAN AKSES 39
A. Menggunakan hak akses (access modifiers) seperti Public, Private, Protected, dan Friend 39
B. Menerapkan enkapsulasi untuk mengontrol aksesibilitas anggota kelas 41
C. Melindungi data dengan menggunakan properti 42
D. Rangkuman 44
E. Latihan 44
KEGIATAN BELAJAR VII - MANAJEMEN MEMORI DAN PENGUMPULAN SAMPAH (GARBAGE COLLECTION) 45
A. Pengertian tentang manajemen memori 45
B. Cara Visual Basic mengelola memori dengan sistem pengumpulan sampah 47
C. Menghindari kebocoran memori dan praktik terbaik 48
D. Rangkuman 50
E. Latihan 50
KEGIATAN BELAJAR VIII - DESAIN BERORIENTASI OBJEK 51
A. Prinsip SOLID dalam desain berorientasi objek 51
B. Menerapkan prinsip-prinsip desain seperti Single Responsibility Principle (SRP) dan Open/Closed Principle (OCP) 52
C. Menggunakan pola desain umum seperti Factory, Singleton,

dan Dependency Injection 53
D. Rangkuman 56
E. Latihan 56
KEGIATAN BELAJAR IX - PENERAPAN PEMROGRAMAN BERORIENTASI OBJEK DALAM APLIKASI NYATA 57
A. Studi kasus: Pembuatan aplikasi sederhana dengan menggunakan konsep pemrograman berorientasi objek 60
B. Pembuatan kelas, pewarisan, polimorfisme, dan antarmuka dalam aplikasi tersebut 59
C. Demonstrasi praktik terbaik dalam mengorganisasi kode dan memisahkan tanggung jawab 62
D. Rangkuman 64
E. Latihan 64
KEGIATAN BELAJAR X - KONSEP PEMROGRAMAN BERORIENTASI OBJEK YANG LEBIH LANJUT 65
A. Lanjutan konsep pemrograman berorientasi objek, seperti delegasi, komposisi, dan berbagai pola desain tingkat lanjut 65
B. Penerapan konsep dalam proyek yang lebih kompleks dan realistis 67
D. Rangkuman 70
E. Latihan 71

DAFTAR PUSTAKA 72
PROFIL PENULIS 73

KEGIATAN BELAJAR I
PENDAHULUAN

DESKRIPSI PEMBELAJARAN

Pada bab ini mahasiswa mempelajari pengenalan tentang konsep pemrograman berorientasi objek dan pengenalan tentang konsep pemograman berorientasi objek. Diharapkan mahasiswa memiliki wawasan dan pemahaman untuk modal dasar mempelajari konsep pemrograman berorientasi objek.

KOMPETENSI PEMBELAJARAN

Setelah mengikuti perkuliahan ini diharapkan mahasiswa dan mahasiswi memiliki pengetahuan dan kemampuan :

1. Mampu menguraikan definisi pemrograman berorientasi objek.
2. Mampu menjelaskan apa itu pemrograman berorientasi objek.
3. Mampu menguraikan lingkup pemrograman berorientasi objek.

A. Pengenalan tentang konsep pemrograman berorientasi objek

Pemrograman Berorientasi Objek (OOP) adalah paradigma pemrograman yang fokus pada konsep objek. Objek merupakan unit utama dalam OOP yang menggabungkan data (atribut) dan perilaku (metode) yang terkait menjadi satu kesatuan. Dalam OOP, program dianggap sebagai kumpulan objek yang berinteraksi satu sama lain untuk mencapai tujuan tertentu.

Pengenalan konsep Pemrograman Berorientasi Objek mencakup prinsip-prinsip dasar berikut:

1. Objek dan Kelas:
   - Objek: Representasi konkret dari entitas di dunia nyata yang memiliki atribut (data) dan metode (perilaku).
   - Kelas: Blueprint atau cetak biru yang mendefinisikan struktur, atribut, dan metode yang dimiliki oleh objek. Objek merupakan instance dari suatu kelas.
2. Enkapsulasi:
   - Konsep enkapsulasi melibatkan pengelompokkan data dan metode yang relevan dalam suatu kelas. Data yang terkait hanya dapat diakses melalui metode yang ditentukan.

- Tujuannya adalah untuk mengurangi kompleksitas dan mengisolasi implementasi internal dari eksternal.

3. Pewarisan:
   - Pewarisan memungkinkan sebuah kelas baru (kelas anak/turunan) mewarisi atribut dan metode dari kelas yang sudah ada (kelas induk/ortu).
   - Ini memungkinkan untuk membuat hierarki kelas dan menghindari pengulangan kode.
4. Polimorfisme:
   - Polimorfisme mengacu pada kemampuan berbagai objek yang berbeda untuk merespons metode-metode yang sama dengan perilaku yang sesuai.
   - Ini memungkinkan penggunaan metode yang sama dengan implementasi yang berbeda.
5. Abstraksi:
   - Abstraksi melibatkan penyederhanaan kompleksitas dengan mengisolasi detail yang tidak relevan dan hanya menampilkan informasi yang penting.
   - Kelas abstrak dan antarmuka (interface) adalah contoh implementasi abstraksi dalam OOP.

Penerapan konsep Pemrograman Berorientasi Objek memiliki beberapa manfaat, termasuk:

1. Modularitas: Program dapat dipecah menjadi bagian-bagian yang lebih kecil (objek) yang lebih mudah dikelola dan dipahami.
2. Reusabilitas: Objek dan kelas yang telah dibuat dapat digunakan kembali dalam proyek-proyek lain.
3. Keterbacaan dan Pemeliharaan: Struktur yang terorganisir dengan baik membuat kode lebih mudah dibaca, dipahami, dan dipelihara.
4. Pengembangan Tim: Pemrograman berorientasi objek memungkinkan tim untuk bekerja secara paralel pada bagian-bagian yang berbeda dari program.

Pemrograman Berorientasi Objek telah menjadi paradigma yang sangat umum dalam pengembangan perangkat lunak modern karena kemampuannya untuk memodelkan dunia nyata dengan lebih akurat dan menghasilkan kode yang lebih terstruktur dan mudah dikelola.

B. Keuntungan dan manfaat dari pemrograman berorientasi objek

Pemrograman Berorientasi Objek (OOP) memiliki banyak keuntungan dan manfaat dalam pengembangan perangkat lunak. Berikut ini adalah beberapa di antaranya:

1. Modularitas: OOP memungkinkan pembagian program menjadi unit-unit yang lebih kecil dan independen yang disebut objek. Ini mempermudah pengelolaan, pemeliharaan, dan pengembangan program yang kompleks.
2. Reusabilitas: Anda dapat menggunakan kembali objek dan kelas yang telah dibuat dalam proyek-proyek berbeda. Ini menghemat waktu dan usaha dalam pengembangan serta meningkatkan efisiensi.
3. Keterbacaan dan Pemeliharaan: OOP mendorong penggunaan struktur terorganisir dan lebih mudah dipahami. Perubahan atau pemeliharaan pada satu bagian program cenderung tidak memengaruhi bagian lainnya.
4. Pengembangan Tim: Dalam proyek besar, berbagai anggota tim dapat bekerja pada bagian-bagian yang berbeda secara paralel, meningkatkan produktivitas dan efisiensi tim.
5. Fleksibilitas: OOP memungkinkan penyesuaian dan perluasan program dengan lebih mudah. Anda dapat menambahkan fungsionalitas baru tanpa mengganggu yang sudah ada.
6. Pengurangan Redundansi Kode: Dengan pewarisan, Anda dapat menciptakan hierarki kelas di mana kelas anak mewarisi sifat-sifat kelas induk, mengurangi duplikasi kode.
7. Enkapsulasi dan Keamanan: OOP memungkinkan Anda untuk mengenkapsulasi data dan metode dalam satu unit yang terisolasi. Ini mengurangi risiko perubahan yang tidak sah atau tidak disengaja.
8. Pengkodean yang Cepat: Anda dapat memanfaatkan kelas-kelas yang sudah ada untuk membangun aplikasi dengan lebih cepat daripada harus membuat semuanya dari awal.
9. Polimorfisme: OOP memungkinkan Anda untuk menggunakan metode yang sama dengan perilaku yang berbeda, meningkatkan fleksibilitas dan kemampuan beradaptasi.
10. Pemodelan Dunia Nyata: OOP memungkinkan Anda untuk mencerminkan struktur dunia nyata ke dalam kode Anda, yang dapat menghasilkan desain yang lebih intuitif dan mudah dimengerti.
11. Pengendalian Kesalahan yang Lebih Baik: Dengan penanganan eksepsi dan mekanisme pelaporan kesalahan, OOP memungkinkan Anda untuk lebih baik dalam mengatasi masalah dan kesalahan dalam program.

12. Pengembangan Berkelanjutan: OOP mendukung pengembangan yang berkelanjutan dan iteratif karena Anda dapat menambahkan fitur baru atau mengubah yang sudah ada tanpa mengubah keseluruhan kode.

Keuntungan ini menjadikan pemrograman berorientasi objek menjadi paradigma yang sangat umum dalam pengembangan perangkat lunak modern.

C. Perbandingan antara pemrograman berorientasi objek dan paradigma lain

Berikut ini perbandingan antara pemrograman berorientasi objek (OOP) dan paradigma pemrograman lain, seperti pemrograman prosedural dan pemrograman fungsional:

1. Pemrograman Berorientasi Objek (OOP):
   a) Fokus Utama: Objek dan hubungan antara objek.
   b) Struktur Utama: Kode dipecah menjadi objek-objek yang memiliki data (atribut) dan metode (perilaku) yang terkait.
   c) Modularitas: Memecah program menjadi unit-unit terpisah untuk memudahkan pengelolaan dan perawatan.
   d) Pewarisan: Mewarisi sifat dan perilaku dari kelas yang sudah ada, memungkinkan pembuatan hierarki kelas.
   e) Polimorfisme: Menggunakan metode yang sama dengan perilaku yang berbeda.
   f) Enkapsulasi: Menyembunyikan detail internal objek dan hanya menampilkan apa yang perlu diketahui oleh pengguna.
   g) Reusabilitas: Memungkinkan penggunaan ulang objek dan kelas dalam proyek-proyek lain.
   h) Keterbacaan dan Pemeliharaan: Struktur terorganisir dan terfokus pada objek membuat kode lebih mudah dipahami dan diubah.

2. Pemrograman Prosedural:
   a) Fokus Utama: Urutan instruksi untuk menyelesaikan tugas.
   b) Struktur Utama: Kode dipecah menjadi fungsi atau prosedur yang mengandung serangkaian instruksi.
   c) Modularitas: Menggunakan fungsi untuk memecah program menjadi bagian-bagian yang lebih kecil.
   d) Variabel: Data yang diubah dalam program menggunakan variabel yang dapat dideklarasikan di berbagai tempat.

e) Keterbacaan dan Pemeliharaan: Mungkin lebih sulit dalam program yang besar karena kurangnya struktur objek.

3. Pemrograman Fungsional:

a) Fokus Utama: Fungsi dan transformasi data.
b) Struktur Utama: Kode dipecah menjadi fungsi yang mengubah data masukan menjadi hasil yang diinginkan.
c) Immutability: Data dianggap tidak berubah setelah dibuat. Operasi fungsi mengembalikan data baru.
d) Pemrograman Fungsional Murni: Tidak ada efek samping dan perubahan state dalam program.
e) Rekursi: Fungsi sering kali diimplementasikan dengan rekursi untuk menghindari perubahan state.
f) Keterbacaan dan Pemeliharaan: Fokus pada transformasi data dapat membuat kode lebih mudah dipahami dalam beberapa kasus.

Perbandingan ini hanya memberikan gambaran umum tentang perbedaan antara paradigma-program paradigma tersebut. Pemilihan paradigma tergantung pada konteks proyek, tujuan, dan preferensi pengembang. Beberapa bahasa pemrograman, seperti Python, mendukung kombinasi dari paradigma berbeda.

D. Penjelasan singkat tentang bahasa Visual Basic sebagai alat pengembangan

Visual Basic (VB) adalah bahasa pemrograman yang dikembangkan oleh Microsoft. Ini adalah salah satu bahasa yang paling terkenal dan digunakan secara luas untuk pengembangan aplikasi berbasis Windows. Berikut adalah penjelasan singkat tentang bahasa Visual Basic:

1. Tujuan Utama:
   Visual Basic awalnya dirancang untuk membuat aplikasi Windows dengan antarmuka grafis yang user-friendly. Namun, seiring waktu, ia juga telah berkembang untuk mendukung pengembangan aplikasi web, layanan, dan banyak domain lainnya.
2. Sintaks Mudah Dibaca:
   Visual Basic menekankan keterbacaan kode dengan menggunakan sintaks yang mirip dengan bahasa manusia dan menggunakan kata-kata yang lebih deskriptif.

3. Event-Driven:
   Salah satu ciri khas Visual Basic adalah pendekatannya yang event-driven, di mana kode dijalankan berdasarkan interaksi pengguna atau peristiwa lainnya.
4. Antarmuka Grafis yang Kuat:
   Visual Basic memungkinkan pembuatan antarmuka grafis dengan cepat menggunakan alat bantu seperti Visual Studio. Ini membuatnya sangat cocok untuk pengembangan aplikasi desktop.
5. Pemrograman Berorientasi Objek:
   Meskipun awalnya Visual Basic lebih bersifat prosedural, versi-versi baru memperkenalkan fitur-fitur pemrograman berorientasi objek yang lebih kuat.
6. Kompatibilitas dengan .NET:
   Visual Basic adalah bagian dari platform .NET, yang berarti bahwa Anda dapat menggunakan banyak perpustakaan dan alat bantu yang ada dalam ekosistem .NET.
7. Rapid Application Development (RAD):
   Visual Basic dirancang untuk pengembangan cepat aplikasi (RAD), yang berarti Anda dapat membuat prototipe atau aplikasi yang lebih kompleks dengan cepat.
8. Komunitas dan Sumber Daya yang Besar:
   Ada komunitas yang besar dan berbagai sumber daya pembelajaran yang tersedia untuk Visual Basic, yang membuatnya lebih mudah untuk belajar dan mengatasi masalah.
9. Pengembangan Aplikasi Desktop dan Web:
   Selain pengembangan aplikasi desktop, dengan pengenalan ASP.NET, Anda juga dapat mengembangkan aplikasi web menggunakan Visual Basic.
10. Integrasi dengan Produk Microsoft:
    Visual Basic terintegrasi dengan produk-produk Microsoft seperti Visual Studio dan banyak teknologi lain yang mendukung pengembangan berbagai jenis aplikasi.

Namun, perlu diingat bahwa penggunaan Visual Basic lebih sering di dalam lingkungan Microsoft dan aplikasi yang berfokus pada platform Windows.

E. RANGKUMAN

Pemrograman Berorientasi Objek (OOP) adalah paradigma pemrograman yang fokus pada konsep objek. Objek merupakan unit utama dalam OOP yang menggabungkan data (atribut) dan perilaku (metode) yang terkait menjadi satu kesatuan. Dalam OOP, program dianggap sebagai kumpulan objek yang berinteraksi satu sama lain untuk mencapai tujuan tertentu. Penerapan konsep Pemrograman Berorientasi Objek memiliki beberapa manfaat.

Pemrograman Berorientasi Objek telah menjadi paradigma yang sangat umum dalam pengembangan perangkat lunak modern karena kemampuannya untuk memodelkan dunia nyata dengan lebih akurat dan menghasilkan kode yang lebih terstruktur dan mudah dikelola.

Pemrograman Berorientasi Objek (OOP) memiliki banyak keuntungan dan manfaat dalam pengembangan perangkat lunak.

Perbandingan memberikan gambaran umum tentang perbedaan antara paradigma-program paradigma tersebut. Pemilihan paradigma tergantung pada konteks proyek, tujuan, dan preferensi pengembang. Beberapa bahasa pemrograman, seperti Python, mendukung kombinasi dari paradigma berbeda

Visual Basic (VB) adalah bahasa pemrograman yang dikembangkan oleh Microsoft. Ini adalah salah satu bahasa yang paling terkenal dan digunakan secara luas untuk pengembangan aplikasi berbasis Windows.

F. LATIHAN

1. Jelaskan Pengenalan tentang konsep pemrograman berorientasi objek ?
2. Jelaskan Keuntungan dan manfaat dari pemrograman berorientasi objek ?
3. Jelaskan Perbandingan antara pemrograman berorientasi objek dan paradigma lain ?

KEGIATAN BELAJAR II
KONSEP DASAR PEMROGRAMAN BERORIENTASI OBJEK

DESKRIPSI PEMBELAJARAN

Pada bab ini mahasiswa mempelajari Konsep dasar pemrograman berorientasi objek. Diharapkan mahasiswa memiliki wawasan dan pemahaman tentang Konsep dasar pemrograman berorientasi objek.

KOMPETENSI PEMBELAJARAN

Setelah mengikuti perkuliahan ini diharapkan mahasiswa dan mahasiswi memiliki pengetahuan dan kemampuan :

1. Mampu mengetahui Konsep dasar pemrograman berorientasi objek.
2. Mampu mengimplementasikan Konsep dasar pemrograman berorientasi objek.

A. Apa itu Objek, Kelas, dan Instance

Dalam Pemrograman Berorientasi Objek (OOP), objek, kelas, dan instance adalah konsep dasar yang membentuk struktur dasar paradigma OOP[1]. Berikut adalah penjelasan singkat tentang ketiganya:

1. Objek:
   Objek adalah representasi konkret dari entitas di dunia nyata atau dari konsep abstrak dalam program. Objek memiliki atribut (data) yang menggambarkan karakteristiknya dan metode (perilaku) yang menjelaskan tindakan atau operasi yang dapat dilakukan oleh objek tersebut. Misalnya, dalam aplikasi perpustakaan, "Buku" bisa menjadi objek dengan atribut seperti judul, penulis, dan tahun terbit, serta metode seperti meminjam dan mengembalikan.
2. Kelas:
   Kelas adalah cetak biru atau template yang mendefinisikan struktur, atribut, dan metode yang akan dimiliki oleh objek. Kelas berfungsi sebagai rancangan untuk menciptakan objek. Misalnya, jika "Buku" adalah objek, "Buku" juga akan menjadi kelas yang mendefinisikan bagaimana objek tersebut akan dibuat dan berperilaku.

3. Instance:
   Instance adalah implementasi nyata dari suatu kelas. Ketika Anda membuat objek dari suatu kelas, Anda menciptakan sebuah instance dari kelas tersebut. Misalnya, jika Anda memiliki kelas "Mobil," setiap mobil nyata yang Anda produksi berdasarkan desain kelas tersebut adalah sebuah instance dari kelas "Mobil."

Secara ringkas:
Kelas adalah cetak biru yang mendefinisikan struktur, atribut, dan metode. **Objek** adalah representasi nyata atau konkret dari kelas dengan atribut dan metode yang sesuai. **Instance** adalah istilah lain untuk objek yang mencerminkan implementasi konkret dari kelas.

Konsep ini memungkinkan pemrogram untuk membuat model dunia nyata dalam kode mereka dengan mengelompokkan data dan perilaku yang terkait menjadi objek yang terorganisir dan mudah dikelola..

B. Enkapsulasi: mengelompokkan data dan metode dalam kelas

Enkapsulasi adalah salah satu prinsip utama dalam Pemrograman Berorientasi Objek (OOP) yang melibatkan pengelompokkan data (atribut) dan metode (perilaku) yang relevan dalam suatu kelas, serta mengatur aksesibilitasnya. Dalam Visual Basic (VB), Anda dapat menerapkan enkapsulasi dengan menggunakan hak akses (access modifiers) dan properti.

Berikut adalah contoh bagaimana enkapsulasi diterapkan dalam VB:

```
Public Class Person
   Private _name As String
   Private _age As Integer
   ' Properti untuk mengakses dan mengubah nilai _name
   Public Property Name() As String
      Get
         Return _name
      End Get
```

```
        Set(ByVal value As String)
            _name = value
        End Set
    End Property

    ' Properti untuk mengakses dan mengubah nilai _age
    Public Property Age() As Integer
        Get
            Return _age
        End Get
        Set(ByVal value As Integer)
            If value >= 0 Then
                _age = value
            Else
                Console.WriteLine("Usia tidak valid.")
            End If
        End Set
    End Property

    ' Metode untuk menampilkan informasi tentang objek Person
    Public Sub DisplayInfo()
        Console.WriteLine("Nama: " & _name)
        Console.WriteLine("Usia: " & _age)
    End Sub
End Class

Module Program
    Sub Main()
        Dim person1 As New Person()
        person1.Name = "John"
        person1.Age = 30
        Dim person2 As New Person()
        person2.Name = "Jane"
        person2.Age = -5
        person1.DisplayInfo()
        person2.DisplayInfo()
```

```
        End Sub
    End Module
```

Dalam contoh di atas, kelas Person mengelompokkan data _name dan _age bersama dengan metode DisplayInfo(). Properti Name dan Age digunakan untuk mengakses dan mengubah data pribadi _name dan _age, sedangkan properti Age memiliki validasi untuk memastikan bahwa usia yang dimasukkan tidak negatif.

C. Pewarisan: menciptakan hierarki kelas dan mewarisi sifat-sifat dari kelas yang lebih tinggi

Pewarisan (inheritance) adalah salah satu konsep penting dalam Pemrograman Berorientasi Objek (OOP) yang memungkinkan Anda untuk membuat hierarki kelas, di mana kelas anak dapat mewarisi sifat dan perilaku dari kelas induk (superclass). Dalam Visual Basic (VB), Anda dapat menciptakan hierarki kelas dan menerapkan pewarisan dengan mudah. Berikut ini adalah contoh penggunaan pewarisan dalam VB:

```
Public Class Animal
   Public Property Name As String
   Public Sub New(ByVal name As String)
      Me.Name = name
   End Sub
   Public Overridable Sub MakeSound()
      Console.WriteLine("Suara hewan...")
   End Sub
End Class

Public Class Dog
   Inherits Animal
   Public Sub New(ByVal name As String)
      MyBase.New(name)
   End Sub
   Public Overrides Sub MakeSound()
      Console.WriteLine("Guk guk!")
   End Sub
```

```
End Class
Public Class Cat
   Inherits Animal
   Public Sub New(ByVal name As String)
      MyBase.New(name)
   End Sub
   Public Overrides Sub MakeSound()
      Console.WriteLine("Meong meong!")
   End Sub
End Class
Module Program
   Sub Main()
      Dim dog As New Dog("Buddy")
      Dim cat As New Cat("Whiskers")
      Console.WriteLine("Nama anjing: " & dog.Name)
      dog.MakeSound()
      Console.WriteLine("Nama kucing: " & cat.Name)
      cat.MakeSound()
   End Sub
End Module
```

Dalam contoh di atas, kita memiliki kelas Animal sebagai kelas induk dengan properti Name dan metode MakeSound(). Kemudian, kita memiliki kelas anak Dog dan Cat yang mewarisi dari kelas Animal menggunakan kata kunci Inherits.

Kelas anak (Dog dan Cat) mewarisi properti dan metode dari kelas induk (Animal). Selain itu, kita dapat menggunakan metode Overrides untuk mengganti perilaku metode yang diwarisi. Dalam contoh tersebut, metode MakeSound() di kelas anak mengganti perilaku yang telah didefinisikan di kelas induk.

Hasil dari program di atas akan mencetak suara yang berbeda untuk anjing dan kucing berdasarkan metode yang di-override dalam kelas anak.

Pewarisan memungkinkan pembuatan hierarki kelas yang memudahkan dalam mengatur dan memahami kode, serta menghindari pengulangan kode yang tidak perlu.

D. Polimorfisme: penggunaan metode yang sama dengan perilaku yang berbeda

Dalam Visual Basic (VB) atau dalam Pemrograman Berorientasi Objek (OOP) secara umum, polimorfisme mengacu pada kemampuan untuk menggunakan metode yang sama dengan perilaku yang berbeda tergantung pada tipe objek yang sedang dioperasikan. Polimorfisme dapat diterapkan melalui pewarisan dan penggunaan metode yang di-override, serta melalui penggunaan antarmuka (interface) dan pengikatan dinamis.

Berikut contoh sederhana penggunaan polimorfisme dalam Visual Basic:

```
Public Class Animal
   Public Overridable Sub MakeSound()
      Console.WriteLine("Suara hewan...")
   End Sub
End Class

Public Class Dog
   Inherits Animal

   Public Overrides Sub MakeSound()
      Console.WriteLine("Guk guk!")
   End Sub
End Class

Public Class Cat
   Inherits Animal

   Public Overrides Sub MakeSound()
      Console.WriteLine("Meong meong!")
   End Sub
End Class

Module Program
```

```
    Sub Main()
        Dim animals() As Animal = {New Dog(), New Cat()}

        For Each animal As Animal In animals
            animal.MakeSound()
        Next
    End Sub
End Module
```

Dalam contoh di atas, kita memiliki kelas Animal sebagai kelas induk dengan metode MakeSound(). Kemudian, kita memiliki kelas anak Dog dan Cat yang mewarisi dari Animal dan meng-override metode MakeSound().

Ketika kita membuat array animals yang berisi objek-objek dari kelas Dog dan Cat, kita menggunakan loop For Each untuk memanggil metode MakeSound(). Meskipun kita memanggil metode yang sama (MakeSound()), perilaku yang dihasilkan berbeda berdasarkan tipe objek yang sedang dioperasikan.

Dalam kasus ini, metode MakeSound() di kelas anak menggantikan perilaku yang telah didefinisikan di kelas induk, menunjukkan konsep polimorfisme, di mana metode yang sama dapat menghasilkan perilaku yang bervariasi sesuai dengan jenis objek yang sedang dioperasikan.

E. Abstraksi: menyembunyikan detail kompleks dari implementasi di luar dunia objek

Abstraksi adalah konsep dalam Pemrograman Berorientasi Objek (OOP) yang melibatkan penyembunyian detail internal dan kompleksitas implementasi suatu objek dari dunia luar, sementara hanya menampilkan informasi penting dan relevan kepada pengguna. Tujuan utama abstraksi adalah menyederhanakan kompleksitas, meningkatkan keterbacaan, dan memungkinkan penggunaan objek tanpa perlu memahami semua detail internalnya.

Dalam Visual Basic (VB) atau OOP pada umumnya, abstraksi dapat dicapai melalui beberapa cara:

1. Penggunaan Akses Modifier:

Menggunakan hak akses (access modifiers) seperti Public, Private, Protected, dan Internal untuk mengontrol visibilitas atribut dan metode dalam kelas. Ini memungkinkan Anda untuk menunjukkan atribut atau metode mana yang boleh diakses dari luar kelas dan yang harus disembunyikan.

2. Penggunaan Properti:

Properti memungkinkan Anda untuk menyembunyikan detail implementasi dari bagaimana data diakses atau diubah. Anda dapat menggunakan properti untuk mengontrol akses ke atribut dan menambahkan logika tambahan saat pengambilan atau pengubahan nilai.

1. Penggunaan Metode:

Menggunakan metode sebagai antarmuka untuk berinteraksi dengan objek. Anda dapat menyembunyikan detail implementasi yang kompleks dalam metode, dan hanya menunjukkan kepada pengguna apa yang dapat dilakukan oleh objek tersebut.

2. Penggunaan Kelas Abstrak:

Kelas abstrak adalah kelas yang tidak dapat diinisialisasi secara langsung, tetapi digunakan sebagai cetak biru untuk kelas-kelas turunannya. Kelas abstrak memungkinkan Anda untuk mendefinisikan metode tanpa memberikan implementasi konkret, meninggalkan implementasi tersebut kepada kelas turunannya.

3. Penggunaan Antarmuka:

Antarmuka adalah kontrak yang menggambarkan metode-metode yang harus ada dalam kelas yang mengimplementasikannya. Ini memungkinkan Anda untuk memisahkan definisi metode dari implementasinya, mengarah pada abstraksi yang lebih besar.

4. Penerapan Design Pattern:

Design pattern seperti Facade dan Adapter juga mendukung abstraksi dengan menyediakan antarmuka yang sederhana dan tersembunyi, menyembunyikan kompleksitas di belakangnya.

Penerapan abstraksi dalam OOP membantu membangun kode yang lebih terorganisir, mudah dimengerti, dan mudah dirawat. Ini juga memungkinkan untuk mengubah detail implementasi tanpa

memengaruhi kode yang menggunakannya, asalkan antarmuka tetap konsisten.

F. Penggunaan konstruktor dan destruktor

Anda Dalam Pemrograman Berorientasi Objek (OOP) dengan Visual Basic (VB), konstruktor dan destruktor adalah metode khusus yang digunakan untuk menginisialisasi dan membersihkan objek. Di VB, konstruktor dikenal sebagai New dan destruktor dikenal sebagai Finalize.

Konstruktor (New):

Konstruktor adalah metode yang akan dijalankan saat objek dibuat (diinisialisasi). Ini digunakan untuk menginisialisasi atribut atau melakukan tindakan lain yang diperlukan saat objek pertama kali diciptakan.

Contoh penggunaan konstruktor dalam VB:

```
Public Class Person
   Public Name As String
   ' Konstruktor
   Public Sub New(ByVal name As String)
      Me.Name = name
      Console.WriteLine("Objek Person dibuat dengan
nama: " & name)
   End Sub
End Class

Module Program
   Sub Main()
      Dim person As New Person("John")
      Console.WriteLine("Nama person: " & person.Name)
   End Sub
End Module
```

Destruktor (Finalize):

Destruktor adalah metode yang akan dijalankan sebelum objek dihapus dari memori. Ini digunakan untuk membersihkan sumber daya yang mungkin digunakan oleh objek sebelum objek dihapus.

Namun, dalam VB, destruktor sekarang tidak umum digunakan karena .NET Framework menyediakan sistem manajemen memori otomatis (garbage collection) yang secara otomatis mengelola pembebasan memori. Namun, Anda masih dapat menerapkan destruktor untuk tindakan pembersihan yang khusus jika diperlukan.
Contoh penggunaan destruktor dalam VB:

```
Public Class Person
   Public Name As String
   ' Konstruktor
   Public Sub New(ByVal name As String)
      Me.Name = name
      Console.WriteLine("Objek Person dibuat dengan
nama: " & name)
   End Sub

   ' Destruktor
   Protected Overrides Sub Finalize()
      Console.WriteLine("Objek Person dihapus dari
memori: " & Me.Name)
      MyBase.Finalize()
   End Sub
End Class
Module Program
   Sub Main()
      Dim person As New Person("John")
      Console.WriteLine("Nama person: " & person.Name)
            person = Nothing ' Membebaskan referensi
            GC.Collect()      ' Meminta garbage collection
(tidak disarankan di dalam kode sehari-hari)
         End Sub
       End Module
```

Dalam contoh di atas, destruktor Finalize akan dijalankan ketika objek Person dihapus dari memori. Namun, perlu diingat bahwa penggunaan destruktor langsung untuk membersihkan sumber daya

tidak disarankan di dalam kode sehari-hari karena manajemen memori otomatis yang dikelola oleh .NET Framework.

G. Rangkuman

Dalam Pemrograman Berorientasi Objek (OOP), objek, kelas, dan instance adalah konsep dasar yang membentuk struktur dasar paradigma OOP . **Kelas** adalah cetak biru yang mendefinisikan struktur, atribut, dan metode. **Objek** adalah representasi nyata atau konkret dari kelas dengan atribut dan metode yang sesuai. **Instance** adalah istilah lain untuk objek yang mencerminkan implementasi konkret dari kelas.

Konsep ini memungkinkan pemrogram untuk membuat model dunia nyata dalam kode mereka dengan mengelompokkan data dan perilaku yang terkait menjadi objek yang terorganisir dan mudah dikelola.

Enkapsulasi adalah salah satu prinsip utama dalam Pemrograman Berorientasi Objek (OOP) yang melibatkan pengelompokkan data (atribut) dan metode (perilaku) yang relevan dalam suatu kelas, serta mengatur aksesibilitasnya. Pewarisan (inheritance) adalah salah satu konsep penting dalam Pemrograman Berorientasi Objek (OOP) yang memungkinkan Anda untuk membuat hierarki kelas, di mana kelas anak dapat mewarisi sifat dan perilaku dari kelas induk (superclass). Dalam Visual Basic (VB), Anda dapat menciptakan hierarki kelas dan menerapkan pewarisan dengan mudah.

Pewarisan (inheritance) adalah salah satu konsep penting dalam Pemrograman Berorientasi Objek (OOP) yang memungkinkan Anda untuk membuat hierarki kelas, di mana kelas anak dapat mewarisi sifat dan perilaku dari kelas induk (superclass).

Dalam Visual Basic (VB) atau dalam Pemrograman Berorientasi Objek (OOP) secara umum, polimorfisme mengacu pada kemampuan untuk menggunakan metode yang sama dengan perilaku yang berbeda tergantung pada tipe objek yang sedang dioperasikan. Polimorfisme dapat diterapkan melalui pewarisan dan penggunaan metode yang di-override, serta melalui penggunaan antarmuka (interface) dan pengikatan dinamis.

H. Latihan

1. Apa itu Objek, Kelas, dan Instance ?
2. Apa yang dimaksud dengan Enkapsulasi: dan Pewarisan ?
3. Apa yang dimaksud dengan Polimorfisme dan Abstraksi ?
4. Bagaimana Penggunaan konstruktor dan destruktor ?

KEGIATAN BELAJAR III
KELAS DAN OBJEK DALAM VISUAL BASIC

DESKRIPSI PEMBELAJARAN

Pada bab ini mahasiswa mempelajari membuat Kelas dan objek Dalam Visual Basic. Diharapkan mahasiswa memiliki wawasan dan pemahaman untuk membuat Kelas dan objek Dalam Visual Basic.

KOMPETENSI PEMBELAJARAN

Setelah mengikuti perkuliahan ini diharapkan mahasiswa dan mahasiswi memiliki pengetahuan dan kemampuan :

1. Mampu membuat membuat Kelas dan objek Dalam Visual Basic.
2. Mampu Mengakses properti dan metode kelas
3. Mampu Mengelola aksesibilitas anggota kelas

A. Membuat Kelas Dalam Visual Basic

Membuat kelas dalam Visual Basic (VB) adalah bagian penting dari Pemrograman Berorientasi Objek (OOP)[2][3]. Kelas adalah cetak biru atau template yang mendefinisikan atribut (data) dan metode (perilaku) yang akan dimiliki oleh objek. Berikut ini adalah cara membuat kelas dalam VB:

```
Public Class Person
   ' Atribut
   Public Name As String
   Public Age As Integer
   ' Konstruktor
   Public Sub New(ByVal name As String, ByVal age As Integer)
      Me.Name = name
      Me.Age = age
   End Sub
   ' Metode
   Public Sub DisplayInfo()
      Console.WriteLine("Nama: " & Name)
      Console.WriteLine("Usia: " & Age)
```

```
    End Sub
End Class
Module Program
    Sub Main()
        ' Membuat objek dari kelas Person
        Dim person1 As New Person("John", 30)
        Dim person2 As New Person("Jane", 25)
        ' Mengakses metode dan atribut
        person1.DisplayInfo()
        person2.DisplayInfo()
    End Sub
End Module
```

Dalam contoh di atas, kita telah membuat kelas Person dengan atribut Name dan Age, serta metode DisplayInfo() untuk menampilkan informasi. Kita juga memiliki konstruktor New untuk menginisialisasi atribut saat objek dibuat. Dalam modul program, kita membuat dua objek dari kelas Person dan menggunakan metode DisplayInfo() untuk menampilkan informasi mereka. Penting untuk diingat bahwa atribut dan metode dalam kelas dapat memiliki hak akses yang berbeda, seperti Public, Private, atau Protected, untuk mengatur visibilitas dan aksesibilitas dari luar kelas.

Membuat kelas adalah cara untuk mengorganisir dan mengelompokkan data dan perilaku yang terkait bersama-sama. Ini membantu dalam memahami dan mengelola kode Anda dengan lebih baik, serta menerapkan konsep-konsep OOP seperti enkapsulasi, pewarisan, dan polimorfisme.

B. Membuat objek dari kelas yang telah dibuat

Membuat objek dari kelas yang telah dibuat dalam Visual Basic (VB) melibatkan langkah-langkah sederhana. Berikut ini adalah contoh cara membuat objek dari kelas yang telah dibuat:

1. Definisikan Kelas:

Pertama-tama, Anda harus memiliki kelas yang telah didefinisikan dengan atribut dan metode yang sesuai.

```
Public Class Person
    Public Name As String
    Public Age As Integer
```

```
        Public Sub New(ByVal name As String, ByVal age As
Integer)
            Me.Name = name
            Me.Age = age
        End Sub

        Public Sub DisplayInfo()
            Console.WriteLine("Nama: " & Name)
            Console.WriteLine("Usia: " & Age)
        End Sub
    End Class
```

2. Membuat Objek:

Setelah Anda memiliki kelas yang sesuai, Anda dapat membuat objek dari kelas tersebut menggunakan pernyataan New.

```
    Module Program
        Sub Main()
            ' Membuat objek dari kelas Person
            Dim person1 As New Person("John", 30)
            Dim person2 As New Person("Jane", 25)

            ' Mengakses metode dan atribut objek
            person1.DisplayInfo()
            person2.DisplayInfo()
        End Sub
    End Module
```

Dalam contoh di atas, kami membuat dua objek dari kelas Person dengan menggunakan pernyataan New. Kemudian, kami menggunakan metode DisplayInfo() pada setiap objek untuk menampilkan informasi yang telah diatur. Penting untuk diingat bahwa Anda dapat membuat sebanyak objek yang Anda butuhkan dari kelas yang sama. Setiap objek akan memiliki instansi unik dari atribut yang didefinisikan dalam kelas. Dengan membuat objek dari kelas, Anda dapat menginstansiasi fungsionalitas dan data yang didefinisikan dalam kelas tersebut dan menggunakannya dalam program Anda.

C. Mengakses properti dan metode kelas

Untuk mengakses properti dan metode kelas pada Visual Basic (VB), Anda harus membuat objek dari kelas tersebut dan kemudian menggunakan objek tersebut untuk mengakses properti dan memanggil metode. Berikut ini adalah contoh bagaimana cara mengakses properti dan metode kelas:

Misalkan kita punya kelas Person seperti ini:

```
Public Class Person
   Public Property Name As String
   Public Property Age As Integer

   Public Sub New(ByVal name As String, ByVal age As
Integer)
      Me.Name = name
      Me.Age = age
   End Sub

   Public Sub DisplayInfo()
      Console.WriteLine("Nama: " & Name)
      Console.WriteLine("Usia: " & Age)
   End Sub
End Class
```

Anda dapat mengakses properti dan metode dalam kode seperti ini:

```
Module Program
   Sub Main()
      ' Membuat objek dari kelas Person
      Dim person1 As New Person("John", 30)

      ' Mengakses properti
      Console.WriteLine("Nama person1: " &
person1.Name)
      Console.WriteLine("Usia person1: " & person1.Age)

      ' Memanggil metode
      person1.DisplayInfo()
   End Sub
End Module
```

Dalam contoh di atas, kita:

1. Membuat objek person1 dari kelas Person.
2. Menggunakan properti Name dan Age untuk mengakses atribut objek.
3. Memanggil metode DisplayInfo() untuk menampilkan informasi dari objek.

Perhatikan bahwa properti diakses seperti atribut biasa, dan metode dipanggil dengan menuliskan nama metode diikuti oleh tanda kurung ().

Dengan mengakses properti dan metode kelas, Anda dapat berinteraksi dengan data dan perilaku yang didefinisikan dalam kelas tersebut dan menggunakannya sesuai kebutuhan program Anda

D. Mengelola aksesibilitas anggota kelas

Dalam Visual Basic (VB) atau dalam Pemrograman Berorientasi Objek (OOP) secara umum, Anda dapat mengelola aksesibilitas anggota kelas dengan menggunakan hak akses (access modifiers). Hak akses mengontrol bagaimana anggota (properti, metode, variabel) dari suatu kelas dapat diakses dan diubah dari luar kelas tersebut. Terdapat beberapa jenis hak akses yang dapat digunakan:

1. Public:

Anggota yang diberi hak akses Public dapat diakses dari mana saja, termasuk dari luar kelas. Ini adalah hak akses paling terbuka.

```
Public Class MyClass
   Public MyProperty As String
End Class
```

2. Private:

Anggota yang diberi hak akses Private hanya dapat diakses oleh kode dalam kelas tersebut. Kode di luar kelas tidak dapat mengakses anggota yang bersifat Private.

```
Public Class MyClass
   Private MyField As Integer
End Class
```

3. Protected:

Anggota yang diberi hak akses Protected dapat diakses dari kelas itu sendiri dan kelas turunannya (inheritance), tetapi tidak dapat diakses dari luar kelas.

```
Public Class MyBaseClass
   Protected MyProtectedProperty As Double
End Class

Public Class MyDerivedClass
   Inherits MyBaseClass

   Sub MyMethod()
      MyProtectedProperty = 5.0
   End Sub
End Class
```

4. Friend:

Anggota yang diberi hak akses Friend hanya dapat diakses oleh kode dalam assembly yang sama. Ini berguna untuk membatasi akses pada level proyek atau modul.

```
Public Class MyClass
   Friend MyField As String
End Class
```

5. Protected Friend :

Kombinasi dari Protected dan Friend. Anggota dengan hak akses ini dapat diakses dari kelas itu sendiri, kelas turunannya, dan dalam assembly yang sama.

```
Public Class MyBaseClass
   Protected Friend MyProtectedFriendProperty As Integer
End Class

Public Class MyDerivedClass
   Inherits MyBaseClass

   Sub MyMethod()
```

```
            MyProtectedFriendProperty = 10
        End Sub
    End Class
```

Dengan menggunakan hak akses, Anda dapat mengontrol tingkat aksesibilitas anggota kelas untuk mencapai enkapsulasi yang baik dan menjaga integritas dan keamanan kode Anda.

E. RANGKUMAN

Membuat kelas dalam Visual Basic (VB) adalah bagian penting dari Pemrograman Berorientasi Objek (OOP). Kelas adalah cetak biru atau template yang mendefinisikan atribut (data) dan metode (perilaku) yang akan dimiliki oleh objek. Membuat kelas adalah cara untuk mengorganisir dan mengelompokkan data dan perilaku yang terkait bersama-sama. Ini membantu dalam memahami dan mengelola kode Anda dengan lebih baik, serta menerapkan konsep-konsep OOP seperti enkapsulasi, pewarisan, dan polimorfisme.

Untuk mengakses properti dan metode kelas pada Visual Basic (VB), Anda harus membuat objek dari kelas tersebut dan kemudian menggunakan objek tersebut untuk mengakses properti dan memanggil metode.

Dalam Visual Basic (VB) atau dalam Pemrograman Berorientasi Objek (OOP) secara umum, Anda dapat mengelola aksesibilitas anggota kelas dengan menggunakan hak akses (access modifiers). Hak akses mengontrol bagaimana anggota (properti, metode, variabel) dari suatu kelas dapat diakses dan diubah dari luar kelas tersebut. Dengan menggunakan hak akses, Anda dapat mengontrol tingkat aksesibilitas anggota kelas untuk mencapai enkapsulasi yang baik dan menjaga integritas dan keamanan kode Anda.

F. LATIHAN

1. Apa yang dimaksud dengan Server Control ?
2. Sebutkan type Server Control ?
3. Bagaimana cara memanipulasi Server Controls di Kode-Behind ?

KEGIATAN BELAJAR IV
PEWARISAN DAN POLIMORFISME

DESKRIPSI PEMBELAJARAN

Pada bab ini mahasiswa mempelajari Pewarisan dan Polimorfisme. Diharapkan mahasiswa memiliki wawasan dan pemahaman untuk membuat Pewarisan dan Polimorfisme.

KOMPETENSI PEMBELAJARAN

Setelah mengikuti perkuliahan ini diharapkan mahasiswa dan mahasiswi memiliki pengetahuan dan kemampuan :

1. Mampu memahami konsep dasar Pewarisan dan Polimorfisme
2. Mampu mempraktekan membuat Pewarisan dan Polimorfisme.

A. Menerapkan pewarisan untuk menciptakan hubungan antara kelas

Pewarisan (inheritance) adalah konsep penting dalam Pemrograman Berorientasi Objek (OOP) yang memungkinkan Anda menciptakan hubungan antara kelas, di mana kelas anak (subclass) dapat mewarisi atribut dan perilaku dari kelas induk (superclass). Di Visual Basic (VB), Anda dapat menerapkan pewarisan dengan mudah. Berikut adalah contoh bagaimana pewarisan digunakan dalam VB:

```
Public Class Animal
   Public Property Name As String

   Public Sub New(ByVal name As String)
      Me.Name = name
   End Sub

   Public Sub MakeSound()
      Console.WriteLine("Suara hewan...")
   End Sub
End Class

Public Class Dog
```

```
    Inherits Animal

    Public Sub New(ByVal name As String)
        MyBase.New(name)
    End Sub

    Public Sub Bark()
        Console.WriteLine("Guk guk!")
    End Sub
End Class

Public Class Cat
    Inherits Animal

    Public Sub New(ByVal name As String)
        MyBase.New(name)
    End Sub

    Public Sub Meow()
        Console.WriteLine("Meong meong!")
    End Sub
End Class

Module Program
    Sub Main()
        Dim dog As New Dog("Buddy")
        Dim cat As New Cat("Whiskers")

        Console.WriteLine("Nama anjing: " & dog.Name)
        dog.Bark()

        Console.WriteLine("Nama kucing: " & cat.Name)
        cat.Meow()
    End Sub
End Module
```

Dalam contoh di atas, kita memiliki kelas Animal sebagai kelas induk dengan properti Name dan metode MakeSound(). Kemudian, kita memiliki kelas anak Dog dan Cat yang mewarisi dari Animal menggunakan kata kunci Inherits.

Kelas anak (Dog dan Cat) mewarisi properti dan metode dari kelas induk (Animal). Selain itu, kita menggunakan metode MyBase untuk memanggil

konstruktor dari kelas induk saat membuat objek kelas anak. Dengan pewarisan, Anda dapat membangun hierarki kelas yang memungkinkan penggunaan kembali kode, mengatur hierarki objek, dan menjaga struktur yang terorganisir.

B. Menggunakan metode yang diwarisi dan menggantikannya (overriding)

Dalam Pemrograman Berorientasi Objek (OOP), Anda dapat menggunakan metode yang diwarisi dari kelas induk (superclass) dan menggantikannya (override) di kelas anak (subclass). Ini memungkinkan Anda untuk mengubah perilaku metode sesuai dengan kebutuhan kelas anak. Di Visual Basic (VB), Anda dapat melakukan overriding dengan mudah. Berikut adalah contoh bagaimana cara menggunakan metode yang diwarisi dan menggantikannya:

```
Public Class Animal
  Public Overridable Sub MakeSound()
    Console.WriteLine("Suara hewan...")
  End Sub
End Class

Public Class Dog
  Inherits Animal

  Public Overrides Sub MakeSound()
    Console.WriteLine("Guk guk!")
  End Sub
End Class

Public Class Cat
  Inherits Animal

  Public Overrides Sub MakeSound()
    Console.WriteLine("Meong meong!")
  End Sub
End Class

Module Program
  Sub Main()
    Dim dog As New Dog()
    Dim cat As New Cat()
```

```
        dog.MakeSound() ' Akan memanggil metode MakeSound() dari kelas Dog
        cat.MakeSound() ' Akan memanggil metode MakeSound() dari kelas Cat
    End Sub
End Module
```

Dalam contoh di atas, kita memiliki kelas Animal sebagai kelas induk dengan metode MakeSound(). Kemudian, kita memiliki kelas anak Dog dan Cat yang mewarisi dari Animal dan menggantikannya menggunakan kata kunci Overrides. Dalam modul program, kita membuat objek dog dan cat dari kelas anak, dan kemudian memanggil metode MakeSound(). Meskipun kita memanggil metode yang sama (MakeSound()), perilaku yang dihasilkan berbeda sesuai dengan metode yang di-override dalam kelas anak.

Penggunaan metode overriding memungkinkan Anda untuk memanfaatkan polimorfisme dan menciptakan perilaku yang berbeda pada objek yang berasal dari kelas yang sama.

C. Menggunakan kata kunci Overrides dan NotOverridable

Kata kunci **Overrides** dan **NotOverridable** adalah bagian penting dari Pemrograman Berorientasi Objek (OOP) dalam Visual Basic (VB). Ini digunakan dalam konteks pewarisan kelas untuk menggantikan atau melarang penggantian metode yang diwarisi dari kelas induk. Mari kita bahas keduanya lebih rinci:

Overrides:

Kata kunci Overrides digunakan dalam kelas anak untuk menandakan bahwa metode dalam kelas anak akan menggantikan (override) metode yang diwarisi dari kelas induk. Ini memungkinkan Anda mengubah perilaku metode sesuai dengan kebutuhan kelas anak.

Contoh:

```
Public Class Shape
   Public Overridable Function CalculateArea() As Double
      Return 0.0
   End Function
End Class

Public Class Circle
```

```
        Inherits Shape

        Public Radius As Double

        Public Sub New(ByVal radius As Double)
            Me.Radius = radius
        End Sub

        Public Overrides Function CalculateArea() As Double
            Return Math.PI * Radius * Radius
        End Function
    End Class
```

NotOverridable:

Kata kunci NotOverridable digunakan dalam kelas induk untuk menandakan bahwa metode tersebut tidak dapat diwarisi atau diubah perilakunya oleh kelas anak. Ini digunakan untuk mencegah penggantian metode tertentu oleh kelas anak.

Contoh:

```
    Public Class Car
        Public NotOverridable Sub StartEngine()
            Console.WriteLine("Mesin mobil dinyalakan.")
        End Sub
    End Class

    Public Class ElectricCar
        Inherits Car

        ' Akan menghasilkan kesalahan karena metode StartEngine diberi
NotOverridable di kelas induk
        Public Overrides Sub StartEngine()
            Console.WriteLine("Mesin mobil listrik dinyalakan.")
        End Sub
    End Class
```

Pada contoh di atas, metode StartEngine dalam kelas Car diberi kata kunci NotOverridable, sehingga tidak dapat diubah perilakunya oleh kelas anak seperti ElectricCar.

Penggunaan kata kunci Overrides dan NotOverridable membantu Anda mengelola pewarisan metode dengan jelas dan mengontrol perilaku metode dalam hierarki kelas.

D. Rangkuman

Pewarisan (inheritance) adalah konsep penting dalam Pemrograman Berorientasi Objek (OOP) yang memungkinkan Anda menciptakan hubungan antara kelas, di mana kelas anak (subclass) dapat mewarisi atribut dan perilaku dari kelas induk (superclass). Di Visual Basic (VB), Anda dapat menerapkan pewarisan dengan mudah. Dalam Pemrograman Berorientasi Objek (OOP), Anda dapat menggunakan metode yang diwarisi dari kelas induk (superclass) dan menggantikannya (override) di kelas anak (subclass). Ini memungkinkan Anda untuk mengubah perilaku metode sesuai dengan kebutuhan kelas anak. Di Visual Basic (VB), Anda dapat melakukan overriding dengan mudah.

Kata kunci **Overrides** dan **NotOverridable** adalah bagian penting dari Pemrograman Berorientasi Objek (OOP) dalam Visual Basic (VB). Ini digunakan dalam konteks pewarisan kelas untuk menggantikan atau melarang penggantian metode yang diwarisi dari kelas induk.

E. Latihan

1. Bagaimana Menerapkan pewarisan untuk menciptakan hubungan antara kelas ?
2. Bagaimana Menggunakan metode yang diwarisi dan menggantikannya (overriding) ?
3. Bagaimana Menggunakan kata kunci Overrides dan NotOverridable ?

KEGIATAN BELAJAR V
ABSTRAKSI DAN ANTARMUKA

DESKRIPSI PEMBELAJARAN

Pada bab ini mahasiswa mempelajari Abstraksi dan Antarmuka. Diharapkan mahasiswa memiliki wawasan dan pemahaman untuk membuat Abstraksi dan Antarmuka.

KOMPETENSI PEMBELAJARAN

Setelah mengikuti perkuliahan ini diharapkan mahasiswa dan mahasiswi memiliki pengetahuan dan kemampuan :

1. Mampu memahami konsep dasar Abstraksi dan Antarmuka
2. Mampu mempraktekan membuat Abstraksi dan Antarmuka.

A. Menerapkan abstraksi untuk membuat kerangka kerja kelas umum

Menerapkan abstraksi untuk membuat kerangka kerja kelas umum adalah strategi yang baik dalam Pemrograman Berorientasi Objek (OOP). Ini memungkinkan Anda untuk membuat kerangka kerja yang dapat diwarisi oleh kelas-kelas turunan untuk membangun fungsionalitas yang lebih khusus. Mari kita lihat contoh penggunaan abstraksi untuk menciptakan kerangka kerja kelas umum dalam Visual Basic (VB):

```
Public MustInherit Class Shape
   Public Property Name As String

   Public Sub New(ByVal name As String)
      Me.Name = name
   End Sub

   Public MustOverride Function CalculateArea() As Double
End Class

Public Class Circle
   Inherits Shape

   Public Radius As Double
```

```
        Public Sub New(ByVal name As String, ByVal radius As
Double)
            MyBase.New(name)
            Me.Radius = radius
        End Sub

        Public Overrides Function CalculateArea() As Double
            Return Math.PI * Radius * Radius
        End Function
    End Class

    Public Class Rectangle
        Inherits Shape

        Public Width As Double
        Public Height As Double

        Public Sub New(ByVal name As String, ByVal width As
Double, ByVal height As Double)
            MyBase.New(name)
            Me.Width = width
            Me.Height = height
        End Sub

        Public Overrides Function CalculateArea() As Double
            Return Width * Height
        End Function
    End Class
```

Dalam contoh di atas, kita membuat kelas abstrak Shape yang memiliki properti Name dan metode abstrak CalculateArea(). Metode ini diberi kata kunci MustOverride, yang mengharuskan kelas-kelas turunannya untuk mengimplementasikan metode ini.

Kemudian, kita membuat dua kelas turunan, yaitu Circle dan Rectangle, yang mewarisi dari Shape. Setiap kelas turunan mengimplementasikan metode CalculateArea() sesuai dengan jenis bentuknya.

Dengan pendekatan ini, Anda dapat membuat kerangka kerja umum (dalam kasus ini, kelas abstrak Shape) yang dapat digunakan untuk menggambarkan berbagai bentuk geometri. Ini membantu dalam penggunaan kembali kode, menyederhanakan pengembangan, dan menjaga konsistensi dalam desain.

B. Membuat antarmuka (interface) untuk menentukan kontrak yang harus diikuti oleh kelas-kelas tertentu

Membuat antarmuka (interface) adalah cara yang sangat berguna untuk menentukan kontrak atau kontrak yang harus diikuti oleh kelas-kelas tertentu dalam Pemrograman Berorientasi Objek (OOP). Antarmuka adalah daftar deklarasi metode dan properti yang harus diimplementasikan oleh kelas yang mengimplementasikan antarmuka tersebut. Mari kita lihat contoh cara membuat antarmuka dalam Visual Basic (VB):

```
Public Interface IShape
   Property Name As String
   Function CalculateArea() As Double
End Interface

Public Class Circle
   Implements IShape

   Public Property Name As String
   Public Radius As Double

   Public Sub New(ByVal name As String, ByVal radius As Double)
      Me.Name = name
      Me.Radius = radius
   End Sub

   Public Function CalculateArea() As Double Implements
IShape.CalculateArea
      Return Math.PI * Radius * Radius
   End Function
End Class

Public Class Rectangle
   Implements IShape

   Public Property Name As String
   Public Width As Double
   Public Height As Double

   Public Sub New(ByVal name As String, ByVal width As Double,
ByVal height As Double)
      Me.Name = name
      Me.Width = width
```

```
            Me.Height = height
        End Sub

        Public Function CalculateArea() As Double Implements
IShape.CalculateArea
            Return Width * Height
        End Function
    End Class
```

Dalam contoh di atas, kita membuat antarmuka IShape dengan properti Name dan metode CalculateArea(). Kemudian, kita membuat dua kelas turunan Circle dan Rectangle yang mengimplementasikan antarmuka IShape. Kedua kelas ini harus mengimplementasikan properti dan metode yang dideklarasikan dalam antarmuka.

Dengan menggunakan antarmuka, Anda dapat membuat kontrak yang harus diikuti oleh kelas-kelas yang mengimplementasikannya. Ini membantu dalam menciptakan struktur yang konsisten dalam kode, memastikan kepatuhan terhadap aturan yang telah ditetapkan, dan memudahkan penggunaan kembali kode.

C. Mengimplementasikan antarmuka dalam kelas

Membuat antarmuka (interface) adalah cara yang sangat berguna untuk menentukan kontrak atau kontrak yang harus diikuti oleh kelas-kelas tertentu dalam Pemrograman Berorientasi Objek (OOP). Antarmuka adalah daftar deklarasi metode dan properti yang harus diimplementasikan oleh kelas yang mengimplementasikan antarmuka tersebut. Mari kita lihat contoh cara membuat antarmuka dalam Visual Basic (VB):

```
    Public Interface IShape
        Property Name As String
        Function CalculateArea() As Double
    End Interface

    Public Class Circle
        Implements IShape

        Public Property Name As String
        Public Radius As Double
```

```
            Public Sub New(ByVal name As String, ByVal radius As Double)
                Me.Name = name
                Me.Radius = radius
            End Sub

            Public Function CalculateArea() As Double Implements
IShape.CalculateArea
                Return Math.PI * Radius * Radius
            End Function
        End Class

        Public Class Rectangle
            Implements IShape

            Public Property Name As String
            Public Width As Double
            Public Height As Double

            Public Sub New(ByVal name As String, ByVal width As Double,
ByVal height As Double)
                Me.Name = name
                Me.Width = width
                Me.Height = height
            End Sub

            Public Function CalculateArea() As Double Implements
IShape.CalculateArea
                Return Width * Height
            End Function
        End Class
```

Dalam contoh di atas, kita membuat antarmuka IShape dengan properti Name dan metode CalculateArea(). Kemudian, kita membuat dua kelas turunan Circle dan Rectangle yang mengimplementasikan antarmuka IShape. Kedua kelas ini harus mengimplementasikan properti dan metode yang dideklarasikan dalam antarmuka.

Dengan menggunakan antarmuka, Anda dapat membuat kontrak yang harus diikuti oleh kelas-kelas yang mengimplementasikannya. Ini membantu dalam menciptakan struktur yang konsisten dalam kode, memastikan

kepatuhan terhadap aturan yang telah ditetapkan, dan memudahkan penggunaan kembali kode.

D. RANGKUMAN

Menerapkan abstraksi untuk membuat kerangka kerja kelas umum adalah strategi yang baik dalam Pemrograman Berorientasi Objek (OOP). Ini memungkinkan Anda untuk membuat kerangka kerja yang dapat diwarisi oleh kelas-kelas turunan untuk membangun fungsionalitas yang lebih khusus. Membuat antarmuka (interface) adalah cara yang sangat berguna untuk menentukan kontrak atau kontrak yang harus diikuti oleh kelas-kelas tertentu dalam Pemrograman Berorientasi Objek (OOP). Antarmuka adalah daftar deklarasi metode dan properti yang harus diimplementasikan oleh kelas yang mengimplementasikan antarmuka tersebut.

Membuat antarmuka (interface) adalah cara yang sangat berguna untuk menentukan kontrak atau kontrak yang harus diikuti oleh kelas-kelas tertentu dalam Pemrograman Berorientasi Objek (OOP). Antarmuka adalah daftar deklarasi metode dan properti yang harus diimplementasikan oleh kelas yang mengimplementasikan antarmuka tersebut.

Dengan menggunakan antarmuka, Anda dapat membuat kontrak yang harus diikuti oleh kelas-kelas yang mengimplementasikannya. Ini membantu dalam menciptakan struktur yang konsisten dalam kode, memastikan kepatuhan terhadap aturan yang telah ditetapkan, dan memudahkan penggunaan kembali kode.

E. LATIHAN

1. Bagaimana Menerapkan abstraksi untuk membuat kerangka kerja kelas umum ?
2. Bagaimana Membuat antarmuka (interface) untuk menentukan kontrak yang harus diikuti oleh kelas-kelas tertentu ?
3. Bagaimana Mengimplementasikan antarmuka dalam kelas ?

KEGIATAN BELAJAR VI
ENKAPSULASI DAN PENGENDALIAN AKSES

DESKRIPSI PEMBELAJARAN

Pada bab ini mahasiswa mempelajari enkapsulasi dan Pengendalian Akses. Diharapkan mahasiswa memiliki wawasan dan pemahaman untuk membuat enkapsulasi dan Pengendalian Akses.

KOMPETENSI PEMBELAJARAN

Setelah mengikuti perkuliahan ini diharapkan mahasiswa dan mahasiswi memiliki pengetahuan dan kemampuan :

1. Mampu memahami konsep dasar enkapsulasi dan Pengendalian Akses
2. Mampu mempraktekan membuat enkapsulasi dan Pengendalian Akses.

A. Menggunakan hak akses (access modifiers) seperti Public, Private, Protected, dan Friend

Hak akses (access modifiers) adalah bagian penting dalam Pemrograman Berorientasi Objek (OOP) yang memungkinkan Anda mengatur tingkat aksesibilitas atribut, metode, atau kelas dalam kode Anda. Di Visual Basic (VB), Anda dapat menggunakan beberapa hak akses seperti Public, Private, Protected, dan Friend. Berikut ini penjelasan singkat tentang masing-masing hak akses:

Public:

Atribut atau metode dengan hak akses Public dapat diakses dari mana saja, baik dari dalam kelas itu sendiri maupun dari luar kelas.

```
Public Class Example
   Public PublicField As Integer
   Public Sub PublicMethod()
      ' ...
   End Sub
End Class
```

Private:

Atribut atau metode dengan hak akses Private hanya dapat diakses dari dalam kelas itu sendiri. Kode di luar kelas tidak dapat mengakses anggota yang diberi hak akses Private.

```
Public Class Example
   Private PrivateField As Integer
   Private Sub PrivateMethod()
      ' ...
   End Sub
End Class
```

Protected:

Atribut atau metode dengan hak akses Protected dapat diakses dari dalam kelas itu sendiri dan oleh kelas turunannya (inheritance), tetapi tidak dapat diakses dari luar kelas.

```
Public Class Example
   Protected ProtectedField As Integer
   Protected Sub ProtectedMethod()
      ' ...
   End Sub
End Class

Public Class DerivedExample
   Inherits Example

   Sub MyMethod()
      ProtectedField = 10
   End Sub
End Class
```

Friend:

Atribut atau metode dengan hak akses Friend hanya dapat diakses oleh kode dalam assembly yang sama. Ini digunakan untuk membatasi akses pada level proyek atau modul.

```
Public Class Example
```

```
        Friend FriendField As Integer
        Friend Sub FriendMethod()
          ' ...
        End Sub
      End Class
```

Dalam kode, Anda dapat menggunakan hak akses ini untuk mengontrol bagaimana atribut dan metode dalam kelas Anda dapat diakses dan digunakan. Hal ini membantu dalam implementasi konsep enkapsulasi dan organisasi yang baik dalam kode Anda.

B. Menerapkan enkapsulasi untuk mengontrol aksesibilitas anggota kelas

.NET Enkapsulasi adalah prinsip penting dalam Pemrograman Berorientasi Objek (OOP) yang melibatkan pengelompokan atribut dan metode bersama-sama dalam suatu kelas serta mengontrol aksesibilitas dari luar kelas. Di Visual Basic (VB), Anda dapat menerapkan enkapsulasi dengan menggunakan hak akses (access modifiers) dan properti (properties) atau metode untuk mengakses atau mengubah nilai atribut.

Berikut adalah contoh penerapan enkapsulasi dalam VB:

```
      Public Class Student
        Private _name As String
        Private _age As Integer

        Public Property Name As String
          Get
            Return _name
          End Get
          Set(ByVal value As String)
            _name = value
          End Set
        End Property

        Public Property Age As Integer
          Get
            Return _age
          End Get
          Set(ByVal value As Integer)
            If value >= 0 Then
              _age = value
```

```
            End If
        End Set
    End Property
End Class

Module Program
    Sub Main()
        Dim student As New Student()

        student.Name = "John"
        student.Age = 25

        Console.WriteLine("Nama: " & student.Name)
        Console.WriteLine("Usia: " & student.Age)
    End Sub
End Module
```

Dalam contoh di atas, kita memiliki kelas Student yang memiliki atribut _name dan _age. Atribut ini diberi hak akses Private untuk menjaga enkapsulasi. Kami menggunakan properti Name dan Age untuk mengakses dan mengubah nilai atribut, dan kami menerapkan validasi pada properti Age untuk memastikan nilai usia selalu positif.

Dengan menggunakan enkapsulasi, Anda membatasi akses langsung ke atribut kelas dan memberikan kontrol atas bagaimana nilai-nilai ini diakses dan diubah. Hal ini membantu dalam menjaga keamanan dan konsistensi data, serta memungkinkan Anda untuk menerapkan validasi atau logika lainnya saat mengakses atau mengubah atribut.

C. Melindungi data dengan menggunakan properti

Enkapsulasi adalah prinsip penting dalam Pemrograman Berorientasi Objek (OOP) yang melibatkan pengelompokan atribut dan metode bersama-sama dalam suatu kelas serta mengontrol aksesibilitas dari luar kelas. Di Visual Basic (VB), Anda dapat menerapkan enkapsulasi dengan menggunakan hak akses (access modifiers) dan properti (properties) atau metode untuk mengakses atau mengubah nilai atribut.

Berikut adalah contoh penerapan enkapsulasi dalam VB:

```
Public Class Student
    Private _name As String
    Private _age As Integer
```

```
        Public Property Name As String
            Get
                Return _name
            End Get
            Set(ByVal value As String)
                _name = value
            End Set
        End Property

        Public Property Age As Integer
            Get
                Return _age
            End Get
            Set(ByVal value As Integer)
                If value >= 0 Then
                    _age = value
                End If
            End Set
        End Property
    End Class

    Module Program
        Sub Main()
            Dim student As New Student()

            student.Name = "John"
            student.Age = 25

            Console.WriteLine("Nama: " & student.Name)
            Console.WriteLine("Usia: " & student.Age)
        End Sub
    End Module
```

Dalam contoh di atas, kita memiliki kelas Student yang memiliki atribut _name dan _age. Atribut ini diberi hak akses Private untuk menjaga enkapsulasi. Kami menggunakan properti Name dan Age untuk mengakses dan mengubah nilai atribut, dan kami menerapkan validasi pada properti Age untuk memastikan nilai usia selalu positif.

Dengan menggunakan enkapsulasi, Anda membatasi akses langsung ke atribut kelas dan memberikan kontrol atas bagaimana nilai-nilai ini diakses dan diubah. Hal ini membantu dalam menjaga keamanan dan konsistensi data, serta memungkinkan Anda untuk menerapkan validasi atau logika lainnya saat mengakses atau mengubah atribut.

D. Rangkuman

Hak akses (access modifiers) adalah bagian penting dalam Pemrograman Berorientasi Objek (OOP) yang memungkinkan Anda mengatur tingkat aksesibilitas atribut, metode, atau kelas dalam kode Anda. Di Visual Basic (VB), Anda dapat menggunakan beberapa hak akses seperti Public, Private, Protected, dan Friend. Enkapsulasi adalah prinsip penting dalam Pemrograman Berorientasi Objek (OOP) yang melibatkan pengelompokan atribut dan metode bersama-sama dalam suatu kelas serta mengontrol aksesibilitas dari luar kelas. Di Visual Basic (VB), Anda dapat menerapkan enkapsulasi dengan menggunakan hak akses (access modifiers) dan properti (properties) atau metode untuk mengakses atau mengubah nilai atribut.

Enkapsulasi adalah prinsip penting dalam Pemrograman Berorientasi Objek (OOP) yang melibatkan pengelompokan atribut dan metode bersama-sama dalam suatu kelas serta mengontrol aksesibilitas dari luar kelas. Di Visual Basic (VB), Anda dapat menerapkan enkapsulasi dengan menggunakan hak akses (access modifiers) dan properti (properties) atau metode untuk mengakses atau mengubah nilai atribut. Dengan menggunakan enkapsulasi, Anda membatasi akses langsung ke atribut kelas dan memberikan kontrol atas bagaimana nilai-nilai ini diakses dan diubah. Hal ini membantu dalam menjaga keamanan dan konsistensi data, serta memungkinkan Anda untuk menerapkan validasi atau logika lainnya saat mengakses atau mengubah atribut.

E. Latihan

1. Bagaimana Menggunakan hak akses (access modifiers) seperti Public, Private, Protected, dan Friend ?
2. Bagaimana Menerapkan enkapsulasi untuk mengontrol aksesibilitas anggota kelas ?
3. Bagaimana Melindungi data dengan menggunakan property ?Apa saja langkah-langkah umum untuk menggunakan Entity Framework dalam proyek ASP.NET ?

KEGIATAN BELAJAR VII
MANAJEMEN MEMORI DAN PENGUMPULAN SAMPAH (GARBAGE COLLECTION)

DESKRIPSI PEMBELAJARAN

Pada bab ini mahasiswa mempelajari Manajemen Memori dan Pengumpulan Sampah. Diharapkan mahasiswa memiliki wawasan dan pemahaman untuk membuat Manajemen Memori dan Pengumpulan Sampah.

KOMPETENSI PEMBELAJARAN

Setelah mengikuti perkuliahan ini diharapkan mahasiswa dan mahasiswi memiliki pengetahuan dan kemampuan :

1. Mampu memahami konsep Manajemen Memori dan Pengumpulan Sampah
2. Mampu mempraktekan membuat Manajemen Memori dan Pengumpulan Sampah.

A. Pengertian tentang manajemen memori

Manajemen memori adalah proses mengelola dan mengontrol alokasi serta penggunaan memori komputer dalam sebuah sistem. Tujuan utamanya adalah untuk memastikan penggunaan memori yang efisien dan efektif oleh program-program yang berjalan, sehingga sistem dapat beroperasi dengan kinerja optimal.

Dalam manajemen memori, beberapa konsep utama yang perlu dipahami meliputi:

1. Alokasi Memori:

Ini adalah proses pemberian blok memori kepada program atau proses yang membutuhkannya. Ini dapat melibatkan alokasi memori fisik secara langsung atau pengaturan dari memori virtual.

2. Dealokasi Memori:

Ini adalah proses membebaskan atau mengembalikan kembali blok memori yang tidak lagi diperlukan oleh program, sehingga memungkinkan memori tersebut digunakan oleh program-program lain.

3. Fragmentasi:

Terdapat dua jenis fragmentasi:

Fragmentasi Internal: Terjadi ketika ada bagian-bagian kecil dari memori yang tidak digunakan dalam suatu blok alokasi.

Fragmentasi Eksternal: Terjadi ketika sejumlah besar memori yang berada dalam potongan-potongan kecil tetapi terpisah-pisah tidak dapat digunakan untuk alokasi yang lebih besar karena posisi dan ukurannya.

Memori Virtual:

Konsep memori virtual memungkinkan program untuk beroperasi seolah-olah memiliki akses ke memori fisik yang lebih besar daripada yang benar-benar ada di sistem. Sistem operasi mengelola pemetaan antara alamat memori virtual dan fisik.

4. Swapping:

Ini adalah proses memindahkan bagian program yang tidak aktif dari memori fisik ke penyimpanan sekunder (seperti hard drive) dan sebaliknya. Ini membantu mengelola kekurangan memori fisik dengan menggunakan ruang penyimpanan yang lebih besar tetapi lebih lambat.

5. Paging dan Segmentasi:

Ini adalah teknik untuk mengelola memori virtual. Dalam paging, memori dibagi menjadi ukuran blok tetap, sedangkan dalam segmentasi, memori dibagi menjadi segmen-segmen berdasarkan jenis data atau fungsi.

6. Garbage Collection:

Ini adalah proses di mana sistem secara otomatis mengidentifikasi dan membebaskan memori yang tidak lagi digunakan oleh program (sampah atau objek yang tidak lagi dapat diakses).

7. Stack dan Heap:

Memori sering dibagi menjadi dua area utama: stack dan heap. Stack digunakan untuk menyimpan variabel-variabel lokal dan informasi pendukung fungsi, sedangkan heap digunakan untuk alokasi memori yang dinamis (objek atau data yang alokasinya tidak diketahui pada waktu kompilasi).

Manajemen memori sangat penting dalam sistem komputer modern, terutama dalam lingkungan multitasking dan multithreading, di mana banyak program atau proses dapat berjalan secara bersamaan. Tujuannya adalah untuk menghindari fragmentasi, mengoptimalkan penggunaan memori, dan mencegah program dari kehabisan memori atau menyebabkan crash sistem.

B. Cara Visual Basic mengelola memori dengan sistem pengumpulan sampah

Visual Basic (VB) adalah bahasa pemrograman yang menggunakan manajemen memori dengan sistem pengumpulan sampah (garbage collection). Sistem pengumpulan sampah adalah proses otomatis yang dilakukan oleh runtime environment atau lingkungan eksekusi untuk mengidentifikasi dan membebaskan memori yang tidak lagi digunakan oleh objek atau data yang tidak dapat diakses. Hal ini membantu mencegah kebocoran memori (memory leaks) dan meningkatkan efisiensi dalam penggunaan memori.

Dalam VB, pengumpulan sampah dilakukan oleh Common Language Runtime (CLR), yang merupakan bagian dari lingkungan .NET Framework. Berikut adalah cara VB mengelola memori dengan sistem pengumpulan sampah:

1. Referensi dan Penghitungan Referensi:

VB menggunakan penghitungan referensi untuk menentukan apakah suatu objek masih digunakan. Setiap kali sebuah objek direferensikan oleh variabel atau objek lain, penghitungan referensi bertambah. Ketika referensi ke objek dihapus atau tidak lagi terjangkau, penghitungan referensi berkurang. Saat penghitungan referensi mencapai nol, objek dianggap tidak lagi digunakan dan dapat dihapus oleh sistem pengumpulan sampah.

2. Generasi dan Pengumpulan Sampah Berbasis Generasi:

Sistem pengumpulan sampah dalam .NET Framework berbasis generasi, yang berarti bahwa objek-objek diatur dalam beberapa "generasi" berdasarkan umur dan aktivitas penggunaan. Objek baru ditempatkan di generasi pertama (gen 0), kemudian pindah ke generasi kedua (gen 1) jika bertahan lebih lama, dan seterusnya. Pengumpulan sampah pada umumnya lebih sering dilakukan pada generasi yang lebih rendah karena objek di generasi yang lebih rendah cenderung memiliki umur lebih pendek.

3. GC.Collect():

Meskipun VB.NET secara otomatis mengatur pengumpulan sampah, Anda juga dapat memicu pengumpulan sampah secara manual menggunakan GC.Collect() jika diperlukan. Namun, dalam kebanyakan kasus, ini tidak diperlukan karena runtime akan mengelola pengumpulan sampah secara efisien.

Penting untuk diingat bahwa dalam sebagian besar situasi, Anda tidak perlu khawatir tentang pengumpulan sampah secara manual dalam VB.NET. Sistem pengumpulan sampah yang terintegrasi dengan baik dalam .NET Framework akan secara otomatis mengelola penghapusan objek yang tidak lagi digunakan, membebaskan Anda dari tugas yang kompleks terkait manajemen memori.

C. Menghindari kebocoran memori dan praktik terbaik

Menggunakan Mencegah kebocoran memori adalah prinsip penting dalam pengembangan perangkat lunak yang stabil dan efisien. Berikut ini adalah beberapa praktik terbaik yang dapat membantu Anda menghindari kebocoran memori dalam pengembangan perangkat lunak menggunakan Visual Basic (VB) atau bahasa pemrograman lain:

1. Gunakan Sistem Pengumpulan Sampah (Garbage Collection):
   Pastikan Anda menggunakan sistem pengumpulan sampah yang disediakan oleh lingkungan eksekusi, seperti Common Language Runtime (CLR) dalam .NET Framework. Jangan mencoba mengelola penghapusan memori secara manual, kecuali jika sangat diperlukan.

2. Hati-Hati dengan Penggunaan Alokasi Memori Dinamis:
   Jika Anda mengalokasikan memori secara dinamis menggunakan konstruktor atau fungsi alokasi, pastikan Anda juga membebaskannya saat tidak lagi diperlukan. Jangan lupakan langkah dealokasi.

3. Hindari Lingering References:
   Pastikan tidak ada referensi yang masih "menggantung" pada objek yang seharusnya sudah tidak digunakan. Pastikan semua referensi terhadap objek dihapus atau di-set ke Nothing jika objek tersebut tidak lagi dibutuhkan.

4. Gunakan Struktur Data yang Benar:
   Pilih struktur data yang sesuai dengan kebutuhan Anda. Jika Anda menggunakan koleksi seperti daftar (lists) atau kamus (dictionaries), pastikan Anda membebaskannya saat selesai digunakan.

5. Hindari Reference Cycles:
   Jika Anda memiliki referensi silang antara objek (misalnya, objek A merujuk ke objek B dan sebaliknya), pastikan sistem pengumpulan sampah dapat mengidentifikasi bahwa objek tersebut tidak lagi digunakan.

6. Gunakan Try-Finally atau Using:
   Saat bekerja dengan objek yang memerlukan pembersihan ekstra, seperti file atau sumber daya jaringan, selalu gunakan blok Try-Finally atau Using untuk memastikan pembersihan yang andal.

7. Uji dan Profil Kode:
   Lakukan pengujian secara menyeluruh untuk mengidentifikasi masalah kebocoran memori. Anda juga dapat menggunakan alat profil untuk menganalisis penggunaan memori dan mengidentifikasi area yang memerlukan perhatian.

8. Hindari Global Variables:
   Penggunaan variabel global dapat menyebabkan masalah dengan kebocoran memori. Sebaiknya gunakan pendekatan yang lebih terstruktur, seperti pengiriman parameter antar metode atau penggunaan kelas yang sesuai.

9. Gunakan Penanganan Pengecualian yang Baik:
   Pastikan bahwa pengecualian atau kesalahan yang terjadi diatasi dengan benar untuk mencegah keadaan tak terduga yang dapat menyebabkan kebocoran memori.

10. Pemantauan dan Peningkatan:
    Perhatikan aplikasi Anda saat berjalan di lingkungan produksi dan pantau penggunaan memori. Lakukan profil dan optimasi jika diperlukan.

Dengan menerapkan praktik terbaik ini dan memiliki pemahaman yang baik tentang manajemen memori, Anda dapat membantu memastikan bahwa aplikasi Anda berjalan dengan lancar tanpa kebocoran memori yang dapat menyebabkan masalah kinerja dan stabilitas.

D. RANGKUMAN

Membuat Model Menggunakan Entity Framework dalam pengolahan data dengan ASP.NET MVC adalah tentang menggunakan Entity Framework (EF) sebagai alat untuk mengakses dan memanipulasi data dalam aplikasi web berbasis ASP.NET MVC. Entity Framework adalah teknologi ORM (Object-Relational Mapping) yang disediakan oleh Microsoft untuk menyederhanakan interaksi dengan basis data relasional.

Dalam konteks pengembangan web dengan ASP.NET MVC, Model mewakili entitas atau objek yang mewakili data yang akan disimpan, diambil, diperbarui, atau dihapus dari basis data. Model ini membantu memisahkan logika bisnis dari akses data. Entity Framework memungkinkan Anda untuk membuat model data berdasarkan struktur tabel dalam basis data dan berinteraksi dengan model ini dalam kode Anda sebagai objek-objek .NET, bukan sebagai kueri SQL.

CRUD Operations adalah singkatan dari Create, Read, Update, dan Delete, yang merupakan empat operasi dasar yang umum digunakan dalam pengolahan data dalam sistem informasi atau aplikasi. Dalam konteks pengembangan web dengan ASP.NET MVC dan menggunakan Entity Framework, CRUD Operations mengacu pada cara Anda melakukan empat operasi dasar ini terhadap data dalam basis data menggunakan model yang dibuat dengan Entity Framework.

Menerapkan validasi data menggunakan Entity Framework dalam pengolahan data dengan ASP.NET MVC merujuk pada cara Anda memastikan bahwa data yang dimasukkan ke dalam basis data adalah valid sebelum diizinkan untuk disimpan. Validasi data adalah langkah penting dalam memastikan integritas dan konsistensi data dalam aplikasi Anda.

E. LATIHAN

1. Bagaimana Membuat Model Menggunakan Entity Framework dalam pengolahan data dengan ASP.NET MVC ?
2. Apakah CRUD Operations ?
3. Bagaimana Menerapkan validasi data menggunakan Entity Framework ?

KEGIATAN BELAJAR VIII
DESAIN BERORIENTASI OBJEK

DESKRIPSI PEMBELAJARAN

Pada bab ini mahasiswa mempelajari cara membuat desain Berorientasi objek. Diharapkan mahasiswa memiliki wawasan dan pemahaman untuk membuat desain Berorientasi objek.

KOMPETENSI PEMBELAJARAN

Setelah mengikuti perkuliahan ini diharapkan mahasiswa dan mahasiswi memiliki pengetahuan dan kemampuan :

1. Mampu membuat desain Berorientasi objek.

A. Prinsip SOLID dalam desain berorientasi objek

Otentikasi Prinsip SOLID adalah kumpulan prinsip desain berorientasi objek yang membantu menghasilkan kode yang lebih bersih, fleksibel, dan mudah dikelola. Prinsip SOLID membantu mengatasi masalah-masalah seperti kekakuan (rigidity), kerapuhan (fragility), dan kerumitan (complexity) dalam kode. Berikut adalah rincian prinsip-prinsip SOLID:

S - Single Responsibility Principle (Prinsip Tanggung Jawab Tunggal):

Sebuah kelas seharusnya hanya memiliki satu alasan untuk berubah. Prinsip ini mendorong pembagian tugas yang jelas dan menghindari kelas yang memiliki terlalu banyak tanggung jawab.

O - Open/Closed Principle (Prinsip Terbuka/Tertutup):

Kode yang baik seharusnya terbuka untuk perluasan (diubah atau diperluas) tetapi tertutup untuk modifikasi (tidak perlu diubah jika fitur baru ditambahkan). Prinsip ini mendorong penggunaan pewarisan dan polimorfisme untuk memperluas fungsionalitas.

L - Liskov Substitution Principle (Prinsip Substitusi Liskov):

Objek dari kelas turunan harus dapat digunakan sebagai pengganti objek dari kelas induk tanpa mengubah kebenaran program. Prinsip ini menjaga hubungan hierarki kelas dan menghindari perilaku yang tidak diharapkan saat melakukan substitusi.

I - Interface Segregation Principle (Prinsip Segregasi Antar Muka):

Klien tidak boleh dipaksa untuk mengimplementasikan metode yang tidak mereka gunakan. Prinsip ini mendorong pembagian antarmuka yang lebih spesifik dan menghindari antarmuka yang terlalu besar.

D - Dependency Inversion Principle (Prinsip Inversi Ketergantungan):

Modul tingkat tinggi tidak boleh bergantung pada modul tingkat rendah, tetapi keduanya harus bergantung pada abstraksi. Prinsip ini mendorong penggunaan injeksi ketergantungan (dependency injection) untuk menghindari ketergantungan yang kaku.

Dengan menerapkan prinsip-prinsip SOLID dalam desain berorientasi objek, Anda dapat menciptakan kode yang lebih mudah diubah, diperluas, dan dikelola. Setiap prinsip SOLID menangani masalah desain yang berbeda, tetapi ketika diterapkan bersama-sama, mereka membantu menciptakan fondasi yang kuat untuk pengembangan perangkat lunak yang berkualitas.

B. Menerapkan prinsip-prinsip desain seperti Single Responsibility Principle (SRP) dan Open/Closed Principle (OCP)

Menerapkan prinsip-prinsip desain seperti Single Responsibility Principle (SRP) dan Open/Closed Principle (OCP) melibatkan pengorganisasian kode Anda dengan cara yang memisahkan tanggung jawab dan memungkinkan perluasan fitur tanpa mengubah kode yang sudah ada. Berikut adalah contoh penerapan kedua prinsip ini dalam sebuah skenario:

Contoh Skenario: Mengelola Pesanan

Kita akan mengembangkan sistem untuk mengelola pesanan dalam sebuah aplikasi e-commerce. Kita akan menerapkan SRP dan OCP dalam desain sistem ini.

Penerapan Single Responsibility Principle (SRP):

Prinsip ini mengatakan bahwa sebuah kelas seharusnya memiliki satu alasan untuk berubah. Mari bagi tanggung jawab sistem ini ke dalam kelas yang berbeda:

Order class: Bertanggung jawab untuk menyimpan informasi pesanan, seperti daftar item, harga, dan data pelanggan.

OrderValidator class: Bertanggung jawab untuk memvalidasi pesanan, seperti memeriksa apakah item tersedia, harga valid, dan informasi pelanggan lengkap.

Penerapan Open/Closed Principle (OCP):

Prinsip ini mengatakan bahwa kode seharusnya terbuka untuk perluasan tetapi tertutup untuk modifikasi. Dalam hal ini, kita akan mempertimbangkan kemungkinan perluasan dengan menambahkan metode pembayaran baru:

PaymentMethod class (kelas abstrak): Mewakili metode pembayaran. Mungkin memiliki metode ProcessPayment yang harus diimplementasikan oleh metode pembayaran spesifik.

CreditCardPayment class (kelas turunan): Mengimplementasikan metode pembayaran dengan kartu kredit. Menggunakan polimorfisme, kita dapat menambahkan kelas-kelas lain seperti PayPalPayment untuk metode pembayaran lain.

Keuntungan Penerapan:

Dengan memisahkan tanggung jawab antara Order dan OrderValidator, kita memisahkan logika bisnis dan validasi, membuat kelas-kelas ini lebih fokus pada tugas masing-masing. Dengan menggunakan PaymentMethod abstrak dan kelas turunan, kita dapat dengan mudah menambahkan metode pembayaran baru tanpa harus mengubah kode yang sudah ada.

Dalam contoh ini, kita menerapkan SRP dengan memisahkan tanggung jawab dan OCP dengan mengizinkan perluasan tanpa modifikasi. Keduanya membantu menciptakan desain yang lebih modular, fleksibel, dan mudah dikelola, yang memungkinkan perubahan dan penambahan fitur tanpa mengganggu seluruh sistem.

C. Menggunakan pola desain umum seperti Factory, Singleton, dan Dependency Injection

Otorisasi Menggunakan pola desain umum seperti Factory, Singleton, dan Dependency Injection (DI) membantu dalam merancang kode yang lebih terstruktur, fleksibel, dan mudah diuji. Di bawah ini, saya akan menjelaskan bagaimana menerapkan masing-masing pola tersebut:

1. Factory Pattern (Pola Pabrik):

Factory Pattern digunakan untuk membuat objek tanpa mengungkapkan logika penciptaan objek secara langsung kepada klien. Ini dapat membantu dalam memisahkan proses pembuatan objek dari kode klien.

Contoh: Pembuatan objek PaymentMethod dengan Factory Pattern

```
Public Class PaymentMethodFactory
  Public Shared Function CreatePaymentMethod(ByVal
paymentType As String) As PaymentMethod
    If paymentType = "CreditCard" Then
      Return New CreditCardPayment()
    ElseIf paymentType = "PayPal" Then
      Return New PayPalPayment()
    Else
      Throw New ArgumentException("Invalid payment type")
    End If
  End Function
End Class
```

2. Singleton Pattern (Pola Singleton):

Singleton Pattern digunakan untuk memastikan bahwa sebuah kelas memiliki hanya satu instance dan menyediakan akses global ke instance tersebut.

Contoh: Implementasi Singleton Pattern

```
Public Class Logger
  Private Shared instance As Logger
  Private Sub New()
    ' Private constructor to prevent instantiation from outside
```

```
    End Sub
    Public Shared Function GetInstance() As Logger
        If instance Is Nothing Then
            instance = New Logger()
        End If
        Return instance
    End Function
    Public Sub Log(ByVal message As String)
        ' Logging logic
    End Sub
End Class
```

3. Dependency Injection (DI):

Dependency Injection adalah pola yang digunakan untuk memasukkan dependensi atau objek yang diperlukan ke dalam kelas melalui konstruktor, metode, atau properti, daripada membuat dependensi di dalam kelas itu sendiri. Ini memungkinkan penggantian komponen dengan mudah dan memungkinkan pengujian yang lebih baik.

Contoh: Penggunaan Dependency Injection dengan Constructor Injection

```
Public Class OrderProcessor
    Private ReadOnly _paymentMethod As PaymentMethod
    Public Sub New(ByVal paymentMethod As PaymentMethod)
            _paymentMethod = paymentMethod
        End Sub
        Public Sub ProcessOrder(ByVal order As Order)
            ' Logic to process order using injected payment method
        End Sub
    End Class
```

Penerapan pola-pola desain ini membantu meningkatkan modularitas, fleksibilitas, dan pemeliharaan dalam kode Anda. Factory Pattern membantu dalam penciptaan objek, Singleton Pattern memastikan instance tunggal, dan Dependency Injection memisahkan ketergantungan dan memudahkan

pengujian. Dengan menggabungkan prinsip SOLID dan pola-pola desain ini, Anda dapat merancang sistem yang lebih kokoh dan mudah dikelola.

D. Rangkuman

Otentikasi Prinsip SOLID adalah kumpulan prinsip desain berorientasi objek yang membantu menghasilkan kode yang lebih bersih, fleksibel, dan mudah dikelola. Prinsip SOLID membantu mengatasi masalah-masalah seperti kekakuan (rigidity), kerapuhan (fragility), dan kerumitan (complexity) dalam kode. Menerapkan prinsip-prinsip desain seperti Single Responsibility Principle (SRP) dan Open/Closed Principle (OCP) melibatkan pengorganisasian kode Anda dengan cara yang memisahkan tanggung jawab dan memungkinkan perluasan fitur tanpa mengubah kode yang sudah ada.

Otorisasi Menggunakan pola desain umum seperti Factory, Singleton, dan Dependency Injection (DI) membantu dalam merancang kode yang lebih terstruktur, fleksibel, dan mudah diuji. Factory Pattern digunakan untuk membuat objek tanpa mengungkapkan logika penciptaan objek secara langsung. Singleton Pattern digunakan untuk memastikan bahwa sebuah kelas memiliki hanya satu instance dan menyediakan akses global ke instance tersebut. Dependency Injection adalah pola yang digunakan untuk memasukkan dependensi atau objek yang diperlukan ke dalam kelas melalui konstruktor, metode, atau properti, daripada membuat dependensi di dalam kelas itu sendiri. Ini memungkinkan penggantian komponen dengan mudah dan memungkinkan pengujian yang lebih baik.

E. Latihan

1. Bagaimana Prinsip SOLID dalam desain berorientasi objek ?
2. Bagaimana Menerapkan prinsip-prinsip desain seperti Single Responsibility Principle (SRP) dan Open/Closed Principle (OCP) ?
3. Bagaimana Menggunakan pola desain umum seperti Factory, Singleton, dan Dependency Injection ?

KEGIATAN BELAJAR IX
PENERAPAN PEMROGRAMAN BERORIENTASI OBJEK DALAM APLIKASI NYATA

DESKRIPSI PEMBELAJARAN

Pada bab ini mahasiswa mempelajari cara penerapan pemrograman berorientasi objek dalam aplikasi nyata. Diharapkan mahasiswa memiliki wawasan dan pemahaman untuk penerapan pemrograman berorientasi objek dalam aplikasi nyata.

KOMPETENSI PEMBELAJARAN

Setelah mengikuti perkuliahan ini diharapkan mahasiswa dan mahasiswi memiliki pengetahuan dan kemampuan :

1. Mampu membuat Konsep Pemrograman Berorientasi objek dalam aplikasi nyata.

A. Studi kasus: Pembuatan aplikasi sederhana dengan menggunakan konsep pemrograman berorientasi objek

Mari kita ambil contoh studi kasus pembuatan aplikasi sederhana untuk manajemen tugas (To-Do List) menggunakan konsep pemrograman berorientasi objek. Aplikasi ini akan memungkinkan pengguna untuk menambahkan, menghapus, dan menampilkan daftar tugas yang perlu dilakukan.

Kelas Task:

Kelas ini akan merepresentasikan setiap tugas dalam To-Do List.

```
Public Class Task
   Public Property Title As String
   Public Property IsDone As Boolean
         Public Sub New(ByVal title As String)
            Me.Title = title
            Me.IsDone = False
         End Sub
      End Class
```

Kelas TodoList:
Kelas ini akan mengelola daftar tugas.

```
Public Class TodoList
   Private tasks As List(Of Task)
   Public Sub New()
      tasks = New List(Of Task)()
   End Sub
   Public Sub AddTask(ByVal title As String)
      Dim newTask As New Task(title)
      tasks.Add(newTask)
   End Sub
   Public Sub RemoveTask(ByVal task As Task)
      tasks.Remove(task)
   End Sub
   Public Sub MarkAsDone(ByVal task As Task)
      task.IsDone = True
   End Sub
   Public Sub DisplayTasks()
      For Each task As Task In tasks
         Console.WriteLine($"{If(task.IsDone,      "[Done]",      "[      ]")}
{task.Title}")
      Next
   End Sub
End Class
```

Program Utama:
Berikut adalah contoh penggunaan kelas-kelas di atas dalam program utama.

```
Module Program
   Sub Main()
      Dim todoList As New TodoList()
            todoList.AddTask("Membuat contoh aplikasi")
            todoList.AddTask("Menggunakan konsep OOP")
            todoList.AddTask("Belajar SOLID Principles")
            todoList.DisplayTasks()
```

```
        Dim taskToRemove As Task = todoList(0)
        todoList.RemoveTask(taskToRemove)

        Dim taskToComplete As Task = todoList(1)
        todoList.MarkAsDone(taskToComplete)

        todoList.DisplayTasks()
    End Sub
End Module
```

Dalam contoh ini, kita telah membuat kelas Task untuk mewakili tugas dan kelas TodoList untuk mengelola daftar tugas. Penggunaan konsep OOP seperti konstruktor, properti, dan metode membantu dalam mengorganisir dan mengelola logika aplikasi dengan lebih baik.

B. Pembuatan kelas, pewarisan, polimorfisme, dan antarmuka dalam aplikasi tersebut

Mari kita perluas studi kasus To-Do List sebelumnya dengan memasukkan konsep pewarisan, polimorfisme, dan antarmuka ke dalamnya. Kita akan menambahkan beberapa jenis tugas yang berbeda dengan pola yang sama dan menggunakan antarmuka untuk memeriksa apakah sebuah tugas sudah selesai atau belum.

Kelas Task (Penggunaan Pewarisan):

```
Public Class Task
    Public Property Title As String
    Public Property IsDone As Boolean

    Public Sub New(ByVal title As String)
        Me.Title = title
        Me.IsDone = False
    End Sub

    Public Overridable Function Display() As String
        Return $"{If(IsDone, "[Done]", "[ ]")} {Title}"
    End Function
```

```
End Class

Public Class NoteTask
   Inherits Task

   Public Sub New(ByVal title As String)
      MyBase.New(title)
   End Sub

   Public Overrides Function Display() As String
      Return $"{If(IsDone, "[Note Done]", "[Note ]")} {Title}"
   End Function
End Class
```

Antarmuka ICompletable (Penggunaan Antarmuka):

```
Public Interface ICompletable
   Property IsDone As Boolean
   Function Display() As String
End Interface
```

Kelas TodoList (Penggunaan Polimorfisme):

```
Public Class TodoList
   Private tasks As List(Of ICompletable)

      Public Sub New()
         tasks = New List(Of ICompletable)()
      End Sub

      Public Sub AddTask(ByVal task As ICompletable)
         tasks.Add(task)
      End Sub

      Public Sub RemoveTask(ByVal task As ICompletable)
         tasks.Remove(task)
      End Sub
```

```
    Public Sub CompleteTask(ByVal task As ICompletable)
        task.IsDone = True
    End Sub

    Public Sub DisplayTasks()
        For Each task As ICompletable In tasks
            Console.WriteLine(task.Display())
        Next
    End Sub
End Class
```

Program Utama:

```
Module Program
    Sub Main()
        Dim todoList As New TodoList()

        Dim task1 As New Task("Create OOP Example")
        Dim task2 As New NoteTask("Study SOLID Principles")

        todoList.AddTask(task1)
        todoList.AddTask(task2)

        todoList.DisplayTasks()

        todoList.CompleteTask(task1)

        todoList.DisplayTasks()
    End Sub
End Module
```

Dalam contoh ini, kita telah menambahkan kelas turunan NoteTask yang mewarisi dari Task, serta mengimplementasikan antarmuka ICompletable untuk menyatakan bahwa kelas tersebut memiliki kemampuan untuk

dinyatakan selesai. Ini memungkinkan kita menggunakan polimorfisme dengan berbagai jenis tugas dalam TodoList.

Penggunaan antarmuka ICompletable memastikan bahwa semua tugas yang ditambahkan ke TodoList memiliki properti IsDone dan metode Display(), memungkinkan kita untuk melakukan operasi tanpa harus mengetahui jenis tugas yang sebenarnya.

Dengan memanfaatkan pewarisan, polimorfisme, dan antarmuka, kita dapat merancang sistem yang lebih fleksibel dan mudah dikelola, dengan kemampuan untuk mengganti jenis tugas atau menambahkan fungsionalitas baru tanpa mengganggu kode yang sudah ada.

C. Demonstrasi praktik terbaik dalam mengorganisasi kode dan memisahkan tanggung jawab

Berikut adalah demonstrasi praktik terbaik dalam mengorganisasi kode dan memisahkan tanggung jawab dalam pengembangan perangkat lunak dengan menggunakan Visual Basic. Kita akan menggunakan contoh aplikasi manajemen kontak sederhana.

Tujuan Aplikasi:

Membuat aplikasi manajemen kontak yang dapat menambahkan, menghapus, dan menampilkan kontak.

Struktur Kode:

1. Models (Folder):
   Contact.vb: Definisi kelas Contact untuk merepresentasikan data kontak.

```
Public Class Contact
    Public Property Name As String
    Public Property Email As String
    Public Property Phone As String
End Class
```

2. Repositories (Folder):
   ContactRepository.vb: Kelas ContactRepository untuk mengelola kontak.

```
Public Class ContactRepository
    Private contacts As List(Of Contact)

    Public Sub New()
        contacts = New List(Of Contact)()
```

```
        End Sub

        Public Sub AddContact(ByVal contact As Contact)
            contacts.Add(contact)
        End Sub

        Public Sub RemoveContact(ByVal contact As Contact)
            contacts.Remove(contact)
        End Sub

        Public Function GetAllContacts() As List(Of Contact)
            Return contacts
        End Function
    End Class
```

3. UI (Folder):
Program.vb: Program utama untuk berinteraksi dengan pengguna.

```
    Module Program
        Sub Main()
            Dim contactRepository As New ContactRepository()

            Dim contact1 As New Contact With {.Name = "John
Doe", .Email = "john@example.com", .Phone = "123-456-7890"}
            Dim contact2 As New Contact With {.Name = "Jane
Smith", .Email = "jane@example.com", .Phone = "987-654-3210"}

            contactRepository.AddContact(contact1)
            contactRepository.AddContact(contact2)

            Dim allContacts As List(Of Contact) =
contactRepository.GetAllContacts()

            For Each contact As Contact In allContacts
                Console.WriteLine($"Name: {contact.Name}, Email:
{contact.Email}, Phone: {contact.Phone}")
            Next
        End Sub
```

```
End Module
```

Penjelasan:

Dalam contoh ini, kode telah diorganisir ke dalam beberapa folder untuk memisahkan tanggung jawab yang berbeda. Struktur folder dan file membantu memisahkan dan mengatur kode sesuai dengan tanggung jawab masing-masing komponen.

1. Folder Models: Berisi definisi kelas Contact yang merepresentasikan data kontak.
2. Folder Repositories: Berisi kelas ContactRepository yang mengelola operasi pada kontak.
3. Folder UI: Berisi program utama Program yang berinteraksi dengan pengguna.

Dengan mengatur kode dalam struktur yang terorganisir, kita membuat kode lebih mudah dimengerti, dikelola, dan memungkinkan penambahan fungsionalitas tanpa mengganggu bagian lain dari kode.

D. RANGKUMAN

Penggunaan antarmuka ICompletable memastikan bahwa semua tugas yang ditambahkan ke TodoList memiliki properti IsDone dan metode Display(), memungkinkan kita untuk melakukan operasi tanpa harus mengetahui jenis tugas yang sebenarnya.

Dengan memanfaatkan pewarisan, polimorfisme, dan antarmuka, kita dapat merancang sistem yang lebih fleksibel dan mudah dikelola, dengan kemampuan untuk mengganti jenis tugas atau menambahkan fungsionalitas baru tanpa mengganggu kode yang sudah ada

E. LATIHAN

1. Buatlah sebuah projek Pemrograman berorientasi objek dalam aplikasi nyata !

KEGIATAN BELAJAR X
KONSEP PEMROGRAMAN BERORIENTASI OBJEK YANG LEBIH LANJUT

DESKRIPSI PEMBELAJARAN

Pada bab ini mahasiswa mempelajari Konsep Pemrograman Berorientasi objek yang lebih lanjut. Diharapkan mahasiswa memiliki wawasan dan pemahaman untuk membuat mempelajari Konsep Pemrograman Berorientasi objek yang lebih lanjut.

KOMPETENSI PEMBELAJARAN

Setelah mengikuti perkuliahan ini diharapkan mahasiswa dan mahasiswi memiliki pengetahuan dan kemampuan :

1. Mampu membuat Konsep Pemrograman Berorientasi objek yang lebih lanjut.

A. Lanjutan konsep pemrograman berorientasi objek, seperti delegasi, komposisi, dan berbagai pola desain tingkat lanjut

Mari kita lanjutkan dengan konsep-konsep lebih lanjut dalam pemrograman berorientasi objek, seperti delegasi, komposisi, dan beberapa pola desain tingkat lanjut. Kami akan menggunakan contoh penerapan dalam pembuatan aplikasi manajemen proyek.

Tujuan Aplikasi:
Membuat aplikasi manajemen proyek yang dapat mengelola tugas, tim, dan risiko.

Delegasi:

Dalam konsep delegasi, suatu objek dapat "mendelegasikan" tanggung jawab atau pekerjaan kepada objek lain. Mari contohkan dengan delegasi tugas manajer proyek kepada anggota tim.

```
Public Class TeamMember
  Public Sub DoTask(ByVal taskName As String)
    Console.WriteLine($"Completing task: {taskName}")
  End Sub
```

```
End Class

Public Class ProjectManager
   Private WithEvents teamMember As TeamMember

   Public Sub New(ByVal member As TeamMember)
      teamMember = member
   End Sub

   Public Sub DelegateTask(ByVal taskName As String)
      Console.WriteLine($"Delegating task: {taskName}")
      teamMember.DoTask(taskName)
   End Sub
End Class
```

Komposisi:

Komposisi adalah cara untuk membangun objek yang lebih kompleks dengan menggabungkan objek yang lebih sederhana. Mari contohkan dengan menggabungkan tugas dan tim dalam proyek.

```
Public Class Task
   Public Property Name As String
End Class

Public Class Team
   Public Property Members As List(Of TeamMember)

   Public Sub New()
      Members = New List(Of TeamMember)()
   End Sub

   Public Sub AddMember(ByVal member As TeamMember)
      Members.Add(member)
   End Sub
End Class

Public Class Project
   Public Property Name As String
   Public Property Tasks As List(Of Task)
```

```
        Public Property Team As Team

        Public Sub New()
          Tasks = New List(Of Task)()
          Team = New Team()
        End Sub
      End Class
```

Pola Desain Tingkat Lanjut:

Selain pola desain yang telah dibahas sebelumnya, ada beberapa pola desain tingkat lanjut yang dapat diterapkan, seperti:

1. Observer Pattern (Pola Pengamat):
Menggunakan pola ini, Anda dapat mengimplementasikan mekanisme notifikasi ketika objek berubah ke banyak objek yang mengamati (observer). Misalnya, memberitahu tim ketika tugas dalam proyek berubah statusnya.

2. Decorator Pattern (Pola Dekorator):
Pola ini memungkinkan penambahan fungsionalitas pada objek tanpa mengubah struktur dasarnya. Misalnya, menambahkan fitur risiko pada tugas proyek.

3. Strategy Pattern (Pola Strategi):
Menggunakan pola ini, Anda dapat mengganti algoritma atau implementasi tertentu dengan mudah tanpa mengubah antarmuka klien. Misalnya, memungkinkan pilihan berbagai metode pengelolaan risiko.

B. Penerapan konsep dalam proyek yang lebih kompleks dan realistis

Mari kita penerapkan konsep-konsep pemrograman berorientasi objek dalam proyek yang lebih kompleks dan realistis menggunakan Visual Basic. Kita akan membuat aplikasi manajemen tugas dengan fitur tambahan, seperti kategorisasi tugas, pengingat, dan pelaporan.

Tujuan Aplikasi:
Membuat aplikasi manajemen tugas yang dapat mengelola tugas dengan berbagai fitur tambahan.

Struktur Kode:

Models (Folder):

Task.vb: Definisi kelas Task untuk merepresentasikan tugas dengan properti dan metode terkait.

```
Public Class Task
   Public Property Title As String
   Public Property DueDate As Date
   Public Property Category As String
   Public Property IsDone As Boolean

   Public Sub CompleteTask()
      IsDone = True
   End Sub

   ' Metode lain yang diperlukan...
End Class
```

Repositories (Folder):

TaskRepository.vb: Kelas TaskRepository untuk mengelola tugas dengan operasi tambah, hapus, dan ambil.

```
Public Class TaskRepository
   Private tasks As List(Of Task)

   Public Sub New()
      tasks = New List(Of Task)()
   End Sub

   Public Sub AddTask(ByVal task As Task)
      tasks.Add(task)
   End Sub

   Public Sub RemoveTask(ByVal task As Task)
      tasks.Remove(task)
   End Sub
```

```
        Public Function GetAllTasks() As List(Of Task)
            Return tasks
        End Function
End Class
```

Services (Folder):
TaskService.vb: Kelas TaskService sebagai lapisan bisnis untuk mengelola tugas dengan fitur tambahan.

```
        Public Class TaskService
            Private taskRepository As TaskRepository

            Public Sub New(ByVal repository As TaskRepository)
                taskRepository = repository
            End Sub

            Public Sub AddTask(ByVal task As Task)
                taskRepository.AddTask(task)
            End Sub

            Public Sub RemoveTask(ByVal task As Task)
                taskRepository.RemoveTask(task)
            End Sub

            Public Function GetAllTasks() As List(Of Task)
                Return taskRepository.GetAllTasks()
            End Function

            ' Metode lain untuk pengingat, pelaporan, dll...
        End Class
```

UI (Folder):
Program.vb: Program utama untuk berinteraksi dengan pengguna, menampilkan tugas, mengatur pengingat, dan menghasilkan laporan.

Pola Desain Tambahan:
Dalam proyek yang lebih kompleks ini, kita juga dapat menerapkan pola desain seperti:

1. Singleton Pattern: Untuk memastikan hanya ada satu instance dari TaskRepository dan TaskService.
2. Decorator Pattern: Misalnya, untuk menambahkan fungsionalitas pengingat pada tugas.
3. Observer Pattern: Misalnya, memberi tahu pengguna saat tugas berubah statusnya.

Kesimpulan:

Dengan mengimplementasikan konsep-konsep seperti kelas, repositori, layanan, dan pola desain, Anda dapat membuat proyek yang lebih kompleks dan realistis dengan kode yang terorganisir, mudah dimengerti, dan dapat dengan mudah dikembangkan lebih lanjut.

C. RANGKUMAN

Selain pola desain yang telah dibahas sebelumnya, ada beberapa pola desain tingkat lanjut yang dapat diterapkan, seperti:

1. Observer Pattern (Pola Pengamat):

Menggunakan pola ini, Anda dapat mengimplementasikan mekanisme notifikasi ketika objek berubah ke banyak objek yang mengamati (observer). Misalnya, memberitahu tim ketika tugas dalam proyek berubah statusnya.

2. Decorator Pattern (Pola Dekorator):

Pola ini memungkinkan penambahan fungsionalitas pada objek tanpa mengubah struktur dasarnya. Misalnya, menambahkan fitur risiko pada tugas proyek.

3. Strategy Pattern (Pola Strategi):

Menggunakan pola ini, Anda dapat mengganti algoritma atau implementasi tertentu dengan mudah tanpa mengubah antarmuka klien. Misalnya, memungkinkan pilihan berbagai metode pengelolaan risiko

Dengan mengimplementasikan konsep-konsep seperti kelas, repositori, layanan, dan pola desain, Anda dapat membuat proyek yang lebih kompleks dan realistis dengan kode yang terorganisir, mudah dimengerti, dan dapat dengan mudah dikembangkan lebih lanjut

D. LATIHAN

1. Buatlah sebuah projek dengan konsep Pemrograman berorientasi objek yang lebih lanjut !

DAFTAR PUSTAKA

[1] K. Kanaki and M. Kalogiannakis, “Introducing fundamental object-oriented programming concepts in preschool education within the context of physical science courses,” *Educ. Inf. Technol.*, vol. 23, no. 6, pp. 2673–2698, 2018, doi: 10.1007/s10639-018-9736-0.

[2] A. Dieter and R. James, “Teaching Object-Oriented Programming Concepts Using Visual Basic . NET,” *J. Inf. Syst. Educ.*, vol. 17, no. 2, pp. 163–169, 2006.

[3] “Application of High Level Programming Language (Visual Basic): A Review,” *J. Sci. Technol.*, no. Volume 5, pp. 90–109, 2020, doi: 10.46243/jst.2020.v5.i5.pp90-109.

PROFIL PENULIS

Nurhadi, S.Kom., M.Kom

Penulis lahir di Bekasi, Jawa Barat bulan Nopember 1978. Penulis menamatkan pendidikan dasar dan menengah di Bekasi, setelah lulus dari STM Negeri 1 Bekasi (sekarang SMK Neger 1 Cikarang Barat) melanjutkan kuliah D3 di STMIK Pranata Indonesia Jurusan Managemen Informatika, kemudian melanjutkan S1 di STMIK Pranata Indonesia Jurusan Sistem informasi. Pada tahun 2015 lulus S2 di universitas Budi Luhur Jakarta Program studi Magister Ilmu Komputer. Memulai karir sebagai IT sejak tahun 2002 sebagai Technikal Support, kemudian menjadi network administrator. Karir di Programmer di mulai sebagai programmer vb 6 dan beralih ke programmer VB.Net dengan databases SQL Server. Saat ini sebagai dosen di STMIK Pranata Indonesia, mengajar untuk mata Kuliah Bahasa Pemrograman 1 dan Bahasa Pemrograman 2. Saat ini penulis aktif sebagai SAP Abap Spesialist. Abap Bahasa pemrograman yang mirip dengan vb.net. Penulis juga aktif membuat video pembelajaran, salah satunya tentang pemrograman visual basic dan SQL server yang dapat di lihat di channel youtube penulis di ***https://bit.ly/PDMVBSQL.***